

BİRLEŞİK DERS KİTABI · DÖRT KAYNAK TEK ÇALIŞMA SİRASINDA

Üç Fazlı BLDC Motoru STM32 ile Sürmek

Elektrik temellerinden six-step komütasyona, pin ve timer ayarlarından Hall geri beslemesine, Nucleo üzerindeki simülasyondan kendi izleme uygulamasına kadar tek sıra hâlinde. Her bölüm bir öncekinin bıraktığı yerden başlar.

PROJE	KARTLAR	SERİ HAT	DURUM
staj_motor_simule staj_motor_simule_g491	NUCLEO-G491RE STM32F401CCUx	LPUART1 / VCP 115200 8-N-1	Komütasyon mantığı güç katı yok

ENERJİ VERMEDEN ÖNCE

Motor ve sürücü modeli doğrulanmadan, giriş doğruluk tablosu okunmadan, dead-time ve fault mekanizması osiloskopla görülmeden güç katına enerji verilmez. İlk denemeler düşük DC gerilimde, akım sınırlamalı kaynakla ve gözetim altında yapılır. Bu kitaptaki bütün pin ve register örnekleri, kullanılan tam parça numarasının datasheet ve reference manual belgeleriyle doğrulanmalıdır.

BÖLÜM 0

Bu kitap nasıl kuruldu, hangi sırayla okunur

Dört ayrı doküman tek bir çalışma sırasına dizildi: önce elektrik ve mikrodenetleyici temelleri, sonra six-step mantığı, sonra sensörler ve yazılım, en sonda gerçekten yapılan uygulamalar ve kendi izleme aracı. Kaynakların bölümleri bu sıraya göre bölündü ve tekrar eden anlatımlar birleştirildi.

KAYNAK	BU KİTAPTAKİ YERİ
Üç Fazlı BLDC Motoru Six-Step Sürme (görsel anlatım rehberi)	Bölüm 1, 5, 8–12, 16 ve Ek A
Pin, Timer, UART ve Hertz (teknik eğitim notu 02)	Bölüm 2, 3, 4
Encoder ve Hall Sensörleri (teknik eğitim notu 01)	Bölüm 6, 7, 13
NUCLEO-G491RE üzerinde komütasyon simülasyonu	Bölüm 13, 14, 15

Zincir: ne, neyi tetikliyor?

İlk okumada formül ezberlenmez. Aşağıdaki beş halka bütün kitabın iskeletidir; her bölüm bir halkayı açar. Bir halka atlanırsa sonraki halka soyut kalır.

1 · YAZILIM MCU PWM üretir Karar verir, enerji vermez	2 · ARAYÜZ Gate driver anahtarları sürer Seviye ve kapı akımı	3 · GÜÇ İnverter U-V-W'yi enerjilendirir 6 MOSFET, DC-link	4 · SIRA Komütasyon tablosu 6 durumu değiştirir Dönen manyetik alan	5 · GERİ BESLEME Hall veya Back-EMF anı söyler Doğru anda komütasyon
--	--	--	---	--

İçindekiler

- 01 · Elektrik ve motor temelleri
- 02 · Pinler: GPIO, alternate function, pull
- 03 · Timer'lar, PWM ve duty cycle
- 04 · UART: TX/RX, baud, 8N1
- 05 · Hertz: saat, PLL, PSC ve ARR
- 06 · Six-step: 60°'lik altı durum

- 07 · Encoder ve Hall: hangi soruya cevap?
- 08 · Hall yazılımı: union, maske, EXTI
- 09 · Güç katı: yarım köprü ve dead-time
- 10 · Sensorless: Back-EMF ve zero crossing
- 11 · Motor kontrol timer'ı ve ADC
- 12 · Yazılım mimarisi, durum makinesi, PI

13 · Uygulama A: NUCLEO-G491RE simülasyonu

14 · Uygulama B: F401 + USB CDC varyantı

15 · Kendi izleme uygulamamız

16 · Gerçek motora geçiş, hata ayıklama, güvenlik

Ek A · Hızlı referans ve formüller

Ek B · Terimler sözlüğü

BÖLÜM 1

Elektrik ve motor temelleri

Dört büyüklük, tek bir soru için: motora ne kadar enerji gidiyor ve nereye ısı olarak dönüyor?

V · volt

Akımı hareket ettiren potansiyel farkı. PWM ile motora giden ortalama gerilim değiştirilir.

I · amper

Sargıdan geçen yük akışı. BLDC'de tork ile doğrudan ilişkilidir.

R · ohm

Akıma karşı koyar, I^2R kaybıyla ısı üretir.

P · watt

Enerji aktarım hızı. DC-link kondansatörü enerji depolar; güç kesilse bile tehlikelidir.

SAYISAL ÖRNEK

$$V = I \cdot R \quad P = I^2 \cdot R$$

Sargı yolunun eşdeğer direnci 0.5Ω ve üzerinden 4 A geçiyorsa yalnız dirençten kaynaklı kayıp $P = 4^2 \cdot 0.5 = 8 \text{ W}$ 'tır. 8 W küçük bir motorda ciddi bir ısıtıcıdır; kilitli veya yanlış komütasyonlu motorun saniyeler içinde ısınmasının sebebi budur.

DC beslemeden dönen manyetik alana

Besleme DC'dir, yönü değişmez. Dönen alanı üreten şey inverter'ın anahtarlama sırasındır. Dört kare bu geçişi kurar: (1) elimizde tek yönlü bir DC bara var, bu haliyle hiçbir şey dönmez. (2) İnverter iki fazı seçer, biri kaynak biri dönüş olur ve sargılarda belirli yönde akım akar. (3) İki sargının alanı toplanır, belirli bir yöne bakan tek bir stator alanı oluşur. (4) Bir sonraki duruma geçilince alan 60° atlar; mıknatıslı rotor bu alanı kovalar. Dönme budur.

BLDC adındaki "DC" neden kafa karıştırır?

FIRÇALI DC

Akım yönünü mekanik fırçalar değiştirir. Motora sabit DC verilir, gerisini metal temaslar halleder. Aşınır, kıvılcım çıkarır.

BLDC

Aynı işi elektronik yapar: fırçanın görevi altı MOSFET ve sizin komütasyon tablonuza geçmiştir. Kaynak DC'dir, sargı gerilimi değil.

Bu kitabı tek cümlede özetleyen ifade şudur: **yazılım fırçadır.**

Motor neden doğrudan STM32 veya ESP'ye bağlanamaz?

Mikrodenetleyici pini 3.3 V mantık seviyesinde ve miliamper mertebesinde akım verir; motor ise amperler ister. Araya gate driver girer: mantık sinyalini MOSFET kapısını hızla açacak akıma ve gerilime çevirir. Tam zincir *MCU* → *driver* → *inverter (6 MOSFET)* → *motor U-V-W* şeklindedir; enerji DC kaynaktan gelir, MCU yalnızca hangi anahtarın ne kadar süre açılacağını söyler. Kontrol yolu (küçük sinyaller) ile güç yolu (motor akımı) birbirinden ayrılır: U-V-W hatları MCU'ya değil inverter çıkışına bağlanır.

TEMEL ZİHİNSEL MODEL

MCU bir orkestra şefidir: hangi anahtar, hangi sırada, ne kadar süre. Sesi çıkaran orkestra ise DC kaynak ve inverter'dır.

Elinizdeki "motor sürücü" hangisi?

Bu sorunun cevabı yazılımınızın ne kadar iş yapacağını belirler.

KART TIPI	SİZDEN İSTER	SIX-STEP'İ KİM YAPAR
Hazır ESC	Throttle PWM, DShot, CAN veya UART komutu	ESC içinde
Entegre 3 faz inverter	3 veya 6 PWM, enable, bazen SPI ayarı	Sizin yazılımınız
Yalnız gate driver	3 veya 6 PWM; harici MOSFET köprüsü gerekir	Yazılımınız + güç kartı
Akıllı motor sürücü	Hız, yön veya tork komutu	Sürücü entegresi

KART GELDİĞİNDE İLK BAKIŞ

Pin adlarında UH/UL, VH/VL, WH/WL veya INHA/INLA görüyorsanız kart doğrudan komütasyon bekleyen bir inverter olabilir. Yalnızca PWM, DIR, EN varsa komütasyon sürücünün içindedir. Kesin karar yalnızca datasheet ve kart şemasıyla verilir.

BÖLÜM 2

Pinler gerçekte nedir?

Bir STM32'de yüzlerce pin vardır, ama hepsi aynı değildir. Pinler çoklu fonksiyon içeren yapılardır: aynı pin hem GPIO çıkışı, hem UART TX, hem timer PWM çıkışı olabilir. Hangi rolü üstleneceği yazılım tarafından seçilir. Pin adı iki parçadan oluşur: port harfi ve pin numarası. PA0 port A'nın 0. pini, PB4 port B'nin 4. pini, PC13 port C'nin 13. pinidir. Aynı pin bir anda yalnızca bir fonksiyonda çalışır ve seçim CubeMX'te yapılır.

ALTERNATE FUNCTION (AF)

UART kullanmak istiyorsanız o pinleri USART moduna çevirmeniz gerekir; GPIO olarak bırakılırsa UART çalışmaz. Pin doğru, mod yanlışsa hiçbir hata mesajı almazsınız — yalnızca veri gelmez.

Ana pin grupları

FONKSİYON	AÇIKLAMA	ÖRNEK PINLER
GPIO	Herhangi bir dijital giriş/çıkış	PA4, PB4, PB5
UART / USART	Seri iletişim (TX, RX)	PA2 (TX), PA3 (RX)
ADC	Analog okuma (0–3,3 V)	PA0, PA1
Timer / PWM	Zamanlama veya hız kontrolü	PA6, PA7
I2C / SPI	Çift telli / dört telli iletişim	PB6–PB7 / PA5–PA7
SWD	Debug ve programlama	PA13, PA14
Boot / reset	Sistem başlatma	BOOT0, NRST

GPIO modları

OUTPUT_PP

Çıkış, push-pull — HIGH/LOW verir

OUTPUT_OD

Çıkış, open-drain — pull-up ile kullanılır

INPUT / ANALOG

Pin durumunu okur / ADC'ye verir

IT_RISING / IT_FALLING

Giriş + kenarda kesme üretir (EXTI)

AF_PP / AF_OD

Alternate function: UART, TIM, SPI gibi çevre birimine bağlanır

Push-pull (PP)

HIGH verildiğinde pin 3,3 V'a, LOW verildiğinde 0 V'a sürülür. Her iki seviye de aktif olarak üretilir. Çoğu durumda kullanılan varsayılan çıkış modu.

Open-drain (OD)

LOW verildiğinde pin 0 V'a çekilir. HIGH verildiğinde pin sürülmez, açık kalır — seviyeyi harici pull-up direnci belirler. I2C gibi "wire-AND" protokollerinde zorunludur.

Pull-up ve pull-down

Bir giriş pini hiçbir yere bağlı değilse *float* durumdadır ve okunan değer belirsizdir; gürültüyle 0 veya 1 görünebilir. Pull-up direnci pini boştayken 3,3 V'a, pull-down direnci 0 V'a çeker. Böylece "basılmamış" hâlin ne okunacağı garanti edilir. Bu projede Hall pinlerinde GPIO_PULLUP vardır: boşta yüksek, sensör bastığında düşük.

HIZLI TEKRAR · BÖLÜM 2

1. Bir pini UART için ayırmayı unutursanız program derlenir mi, çalışır mı?
2. Open-drain çıkış neden tek başına HIGH üretemez?
3. Pull-up'lı bir butonda basılmamış hâl hangi seviyedir?

BÖLÜM 3

Timer'lar, PWM ve duty cycle

Timer, içinde bir sayaç bulunan çevre birimidir. Saat sinyalinin beslenir, belirli aralıklarla otomatik sayar ve CPU'yu meşgul etmez. Sayaç hedef değere ulaştığında kesme üretir veya bir çıkış pininin seviyesini değiştirir. Timer'ın değeri, işi yazılım olmadan yapmasıdır: CPU yalnızca sonucu okur.

saat → timer sayacı 0 → 1 → 2 → ... → CNT = ARR → kesme veya PWM değişimi

Timer modları

Basic	Sadece sayar, kesme üretir
PWM	Belirli oranda aç/kapa sinyali üretir
Encoder	A/B sinyallerini donanımda sayar
Input capture	Sinyalin değişim anını/süresini ölçer
One-pulse	Tetiklenince tek darbe üretir
Watchdog	Program takılırsa sistemi sıfırlar

G4 ailesindeki timer'lar

TIM1, TIM8, TIM20	Gelişmiş: PWM + komütasyon + encoder → motor kontrolü
TIM2 ... TIM5	Genel amaçlı, 32/16-bit → pozisyon ve PWM
TIM6, TIM7	Basic → periyodik kesme
LPTIM	Düşük güç timer'ı → uyku modunda zaman
SysTick	Çekirdek zamanlayıcısı → HAL_GetTick()

PWM ve duty cycle: "ne kadar" ayarı

Anahtar ya tam açık ya tam kapalıdır; arada bir değer yoktur. Ortalama gerilim, açık kalma oranıyla ayarlanır. Frekans sabit tutulur, yalnızca açık kalma süresi değişir: %25 duty yavaş dönüş ve düşük tork, %75 duty yükselmiş ortalama gerilim demektir. Periyot = 1 / frekans; duty cycle periyodu değiştirmez, yalnızca içindeki açık süreyi değiştirir.

SAYISAL ÖRNEK

$$D = t_{on} / T \quad V_{ort} \approx D \cdot V_{bus}$$

24 V bus, 20 kHz PWM (T = 50 µs), t_{on} = 20 µs ise D = 0.40 ve V_{ort} ≈ 9.6 V. Buradan RPM çıkmaz: gerçek hız yük, motor sabitleri, sürtünme ve besleme çökmesine bağlıdır. Bu yüzden Bölüm 12'de hız ölçülüp geri beslenir.

BU PROJEDE

Motor PWM ile değil, GPIO aç/kapa ile sürülüyor: her 6-step adımında ilgili pinler elle HIGH/LOW yapılıyor. Gerçek motor kontrolünde sürücüye PWM verilir ve duty cycle doğrudan motor hızını belirler.

BÖLÜM 4

UART: TX/RX, baud rate ve 8N1

UART, iki cihaz arasında iki tel üzerinden seri veri taşır. TX pini veri gönderir, RX pini veri alır. Aynı bir saat teli gerektirmez — bu yüzden "asenكرون"dur. Bağlantının tek kritik kuralı çaprazlamadır: A cihazının TX'i, B cihazının RX'ine gider. Ayrıca iki taraf ortak bir GND paylaşmak zorundadır.

Ortak GND olmadan seviyeler referanssız kalır ve iletişim çalışmaz. Bu şarttır, tercih değil.

STM32 (cihaz A) ESP32 / PC (cihaz B) TX · PA2 —————→ RX RX · PA3 ←———— TX GND ————— GND TX → RX ve RX → TX: çapraz

Baud rate ve veri çerçevesi

Baud, saniyede gönderilen bit sayısıdır. Alıcı ve verici aynı baud ile çalışmak zorundadır; aksi hâlde veri anlamsız karakterlere dönüşür. Bu projede 115200 baud kullanılıyor; düşük baud değerleri daha yavaştır, ancak uzun ve gürültülü hatlarda daha güvenilirdir. Bir karakter — örneğin 'A' = 0x41 = 0100 0001 — hat üzerine şu sırayla çıkar: bir start biti, sekiz veri biti, isteğe bağlı parity biti ve bir ya da iki stop biti. WordLength 8 · Parity None · StopBits 1 birleşimi **8N1** olarak yazılır.

```
// Senkron – gönderme bitene kadar bekler (bloke edici)
HAL_UART_Transmit(&hlpuart1, data, size, timeout);

// Kesme tabanlı – göndermeye baslar, arka planda tamamlar
HAL_UART_Transmit_IT(&hlpuart1, data, size);

// DMA tabanlı – en verimli, büyük veriler için
HAL_UART_Transmit_DMA(&hlpuart1, data, size);
```

NEDEN LPUART1?

G491 projesinde USART yerine LPUART1 (Low Power UART) kullanılıyor. Aynı işi yapar, fakat düşük güç tüketir; PA2 = TX, PA3 = RX pinlerine bağlıdır. Düşük güç mimarisi nedeniyle üst baud sınırı daha düşüktür, 115200 civarında kalır.

HIZLI TEKRAR · BÖLÜM 4

1. TX–TX bağlarsanız ne olur?
2. Bir taraf 9600, diğeri 115200 baud ile çalışırsa terminalde ne görürsünüz?
3. 8N1 çerçevesinde bir karakter için hat üzerinde toplam kaç bit gider?

BÖLÜM 5

Hertz: frekans, periyot, saat ve PLL

Hertz, bir olayın saniyede kaç kez tekrarlandığını gösterir. Frekans tekrar sayısıdır, periyot ise tek bir tekrarın süresidir; ikisi birbirinin tersidir. $1 \text{ Hz} \rightarrow 1 \text{ s}$, $10 \text{ Hz} \rightarrow 100 \text{ ms}$, $100 \text{ Hz} \rightarrow 10 \text{ ms}$, $170 \text{ MHz} \rightarrow 5,88 \text{ ns}$. Zamanlama hesaplarında hemen her adım bu iki satır arasında gidip gelir.

STM32 hızlı çalışabilmek için bir saat sinyali kullanır ve bu sinyal çekirdekten çevre birimlerine kadar bütün bloklara dağıtılır: SYSCLK $\rightarrow$ CPU, AHB bus (GPIO ve bellek), APB1 ve APB2 busları (timer'lar, UART, ADC). Bir çevre birimi çalışmıyorsa ilk kontrol edilecek şey saatinin açık olup olmadığıdır.

Timer'da Hz: prescaler ve auto-reload

Timer sayacı, timer saatinden beslenir; saat ne kadar hızlıysa sayaç o kadar hızlı artar. Prescaler (PSC) gelen frekansı böler, auto-reload (ARR) sayacın hangi değerde sıfırlanacağını belirler. İkisi birlikte kesme aralığını verir. PSC ve ARR register'larına yazılan değer, istenen bölme oranından bir eksiktir: 170 bölme için $PSC = 169$.

170 MHz timer saati $\rightarrow PSC = 169$ ($\div 170$) $\rightarrow 1 \text{ MHz}$ ($1 \mu\text{s}$) $\rightarrow ARR = 999$ (1000 sayım) $\rightarrow 1 \text{ ms}$ kesme

KESME FREKANSI VE CENTER-ALIGNED PWM

$$f_{\text{kesme}} = f_{\text{timer}} / ((PSC + 1) \times (ARR + 1))$$

$$f_{\text{PWM,center}} = f_{\text{TIM}} / [2(PSC+1)(ARR+1)]$$

$f_{\text{TIM}} = 170 \text{ MHz}$, $PSC = 0$, $ARR = 4249$ için $f_{\text{PWM}} = 170.000.000 / (2 \cdot 1 \cdot 4250) = 20 \text{ kHz}$. $ARR = 4249$ aynı zamanda duty çözünürlüğünüzdür: CCR değerini 0–4249 arasında ayarlarsınız, yani yaklaşık %0.024 adımlarla.

PLL ve saat kaynakları

CPU yüksek frekansa, iç ya da dış osilatörden gelen düşük frekansı PLL ile çarparak ulaşır. PLL içinde önce bir bölme (M), sonra bir çarpma (N), sonra tekrar bir bölme (R veya P) yapılır. Kaynak olarak HSI (iç) veya HSE (dış kristal) seçilebilir.

G491 · 170 MHz

F401 · 84 MHz (Bölüm 14)

HSI = 16 MHz PLL giriş = $16 / 4 = 4$ MHz (PLLM = DIV4) VCO = $4 \times 85 = 340$ MHz (PLLN = 85) SYSCLK = $340 / 2 = 170$ MHz (PLLR = DIV2)

HSI = 16 MHz PLL giriş = $16 / 16 = 1$ MHz (M = 16) VCO = $1 \times 336 = 336$ MHz (N = 336) SYSCLK = $336 / 4 = 84$ MHz (P = 4) USB 48M = $336 / 7 = 48$ MHz (Q = 7)

KAVRAM	ANLAMI	BU PROJEDE
SYSCLK	İşlemci saat hızı	170 MHz (G491) / 84 MHz (F401)
HAL_GetTick()	SysTick'in saydığı milisaniye	Adım zamanlaması (now_ms)
UART baud	Seri veri hızı	115200 baud
Elektriksel RPM	Elektriksel dönüş hızı	6 adım $\times$ komütasyon frekansı

BÖLÜM 6

Six-step: 60 derecelik altı durum

Buradaki 60 derece, motorun gövdesini altı dilime bölmek değildir. Manyetik alanın bir elektriksel turundaki sektördür; karışıklığın kaynağı burasıdır. Bir elektriksel tur altı eşit sektöre bölünür ($6 \times 60^\circ = 360^\circ$ elektriksel) ve her sektör bir komütasyon durumudur.

Elektriksel açı ile mekanik açı aynı şey değil

Rotorda kutup çifti sayısı (p) kadar manyetik desen tekrarı vardır. $p = 7$ olan bir motorda mil bir kez dönerken yazılım 7 elektriksel tur, yani 42 kez durum değiştirir. Yazılımın gördüğü hız, milin hızı değildir.

$$\theta_e = p \cdot \theta_m$$

$p = 7$ için mil 1° dönerse alan 7° elektriksel ilerler. Ters yönden bakınca: bir 60° elektriksel sektör, mekanikte $60/7 \approx 8.57^\circ$ 'dir. Yazılımda kontrol edilen şey milin açısı değil, bu altı elektriksel durumdur.

Altı durum: her adımda hangi faz ne yapıyor?

Kural her adımda aynı: bir faz pozitif (kaynak, PWM ile sürülür), bir faz negatif (dönüş yolu, GND), üçüncü faz boşta (iki anahtar da kapalı; Back-EMF tam burada ölçülür). Altı adım bu üç rolün altı farklı dağılımıdır.

ADIM	FAZ U / A	FAZ V / B	FAZ W / C	AKIM YOLU
1	HIGH (+1)	LOW (-1)	FLOAT (0)	$U \rightarrow V$
2	HIGH (+1)	FLOAT (0)	LOW (-1)	$U \rightarrow W$
3	FLOAT (0)	HIGH (+1)	LOW (-1)	$V \rightarrow W$
4	LOW (-1)	HIGH (+1)	FLOAT (0)	$V \rightarrow U$
5	LOW (-1)	FLOAT (0)	HIGH (+1)	$W \rightarrow U$
6	FLOAT (0)	LOW (-1)	HIGH (+1)	$W \rightarrow V$

İLERİ ▶ 1 → 2 → 3 → 4 → 5 → 6 → 1 · GERİ ◀ 6 → 5 → 4 → 3 → 2 → 1 → 6. Sıra tersine çevrilirse motor ters döner. Gerçek faz eşlemesi donanıma göre doğrulanmalıdır.

SIRALAMA HATASININ BEDELİ

Faz isimleri veya sensör eşlemesi yanlışsa motor dönmez; titrer, ısınır ve aşırı akım çeker. Bu yüzden tablo, enerji verilmeden logic analyzer ile doğrulanır (Bölüm 16, Aşama 3).

Bir durumu inverter anahtarlarına çevirmek

Yazılım katmanı yalnız soyut faz durumunu üretir: "U pozitif · V negatif · W boş". Bu satır altı anahtara çevrilir — U üst anahtar PWM ile sürülür, V alt anahtar açık tutulur, W'nin iki anahtarı da kapalıdır. Tablo GPIO komutuna çevrilmeden önce sürücünün doğruluk tablosu okunur: 3-PWM modunda yön/enable ayrıdır ve dead-time sürücüdedir, 6-PWM modunda altı çıkışı siz sürersiniz, aktif-low sürücülerde polarite terstir.

GPIO İLE RASTGELE ANAHTARLAMA YAPMAYIN

High-side ve low-side pinlerini sıradan GPIO yazmalarıyla farklı zamanlarda değiştirmek geçici shoot-through üretir. Motor kontrol timer'ının complementary PWM, preload/COM ve break özellikleri kullanılır. Tüm çıkışların güvenli başlangıç durumu, MCU reset olurken de tanımlı olmalıdır.

Yön, hız ve tork üç farklı şeydir

Yön

Komütasyon sırası belirler: 1→2→3 ileri, 6→5→4 geri. Duty ile ilgisi yoktur.

Hız

Komütasyon sıklığı ve ortalama gerilimle ilişkilidir. Kesin hız için ölçülen RPM geri beslenir.

Tork

Temel olarak faz akımıyla ilişkilidir. Kilitli motor yüksek akım çeker ama dönmez.

Sadece duty vererek sabit hız elde edilmez: yük değişince hız da değişir. Sabit RPM istiyorsanız hızı ölçmek ve duty'yi kontrolcüye ayarlatmak zorunludur.

BÖLÜM 7

Encoder ve Hall: hangi soruya cevap veriyorlar?

Bir mikrodenetleyici motor milini göremez; yazılım tarafında elinizde yalnızca pinlerdeki gerilim seviyeleri vardır. Dönme hareketinin önce elektriksel bilgiye çevrilmesi gerekir, bu çeviriyi sensör yapar. İki sensörü karıştırmamanın en kısa yolu, hangi soruya cevap verdiklerini hatırlamaktır.

ENCODER'IN SORUSU

"Mil ne kadar döndü ve ne hızla dönüyor?"

HALL'IN SORUSU

"Rotor şu anda hangi elektriksel sektörde; sıradaki komütasyon ne olmalı?"

Encoder: A/B kanalı ve x4 sayım

Artımlı encoder, mil döndükçe darbe üretir. Optik modelde yarıklı bir disk LED ile fotodedektör arasından geçer; dönüş devam ettikçe çıkış sürekli kenar üretir. Çıkışın HIGH mi LOW mu olduğu devreye bağlıdır ve tek başına anlam taşımaz — önemli olan kararlı ve sayılabilir kenarların oluşmasıdır. Artımlı encoder yalnızca değişimi bilir: güç kesilirse sayaç sıfırlanır, mutlak konum için referans alma (homing) gerekir.

A ve B aynı frekanstadır, fakat aralarında yaklaşık 90° elektriksel faz farkı vardır; bu düzene *quadrature* denir. Yön bilgisi tek kanaldan çıkmaz: hangi kanalın kenarının önce geldiğine bakılır. "A > B" bir büyüklük karşılaştırması değildir; doğru ifade, A'nın faz olarak B'den önde olmasıdır. A ile B'nin kabloları yer değiştirirse sayım yönü tersine döner.

Tek bir A periyodunda dört kenar bulunur: A yükselir, B yükselir, A düşer, B düşer. Timer bu kenarlardan birini sayarsa x1, ikisini x2, dördünü x4 çözünürlük elde edilir. Aynı encoder, yalnızca sayım kipini değiştirerek dört kat çözünürlük verebilir. Z (index) kanalı varsa turda bir darbe üretir; başlangıç referansı içindir, A/B'nin yerine geçmez.

ÖRNEK · ÇÖZÜNÜRLÜK

$$N_{\text{tur}} = 4 \times 600 = 2400 \text{ count/tur} \quad \cdot \quad \theta_{\text{count}} = 360^\circ / 2400 = 0,15^\circ$$

PPR ve CPR tanımları üreticiden üreticiye değişir. Datasheet'teki değerin periyot mu kenar mı olduğu mutlaka kontrol edilmelidir; aksi halde ölçülen açı dört kat yanlış olur.

"Ölü encoder / canlı encoder" standart terimler değildir. Doğru ayrım şudur: **mekanik artımlı encoder** kontakları açıp kapatır, kontak sıçraması (bounce) için debounce gerekir; **aktif optik/manyetik encoder** besleme ister ve push-pull, open-collector veya diferansiyel elektronik çıkış üretir. Open-collector çıkışta harici pull-up gerekir ve 5 V çıkışın STM32 pinine doğrudan uygun olduğu varsayılmaz.

STM32 timer ile encoder okuma

STM32 timer'larının encoder arayüzü A ve B kanallarını donanımda izler ve sayacı yönüne göre artırıp azaltır. CPU her kenarda kesmeye girip yön hesabı yapmaz; yazılımın işi yalnızca sayacı belirli aralıklarla okumaktır. 10 000 count/s hızda yazılım kesmesi CPU'yu boğar, timer ise hiç yük bindirmez. CubeMX'te uygun timer için Encoder Mode seçilir, iki pin Alternate Function olarak atanır, sayaç periyodu belirlenir ve gürültülü ortamda dijital giriş filtresi ayarlanır.

```
HAL_TIM_Encoder_Start(&htim2, TIM_CHANNEL_ALL);

uint16_t simdiki = (uint16_t)__HAL_TIM_GET_COUNTER(&htim2);
int16_t delta = (int16_t)(simdiki - onceki); // wrap-around ile uyumlu
onceki = simdiki;
```

HIZ FORMÜLÜ

$$\text{RPM} = (\Delta \text{count} / N_{\text{tur}}) \times (60 / \Delta t)$$

Hız daima sabit bir örnekleme aralığında hesaplanır; Δt kayarsa hız da kayar.

SAYISAL ÖRNEK

2400 count/tur encoder, $\Delta t = 0,01$ s ve bu sürede $\Delta \text{count} = 40$ ise:

$$\text{RPM} = (40 / 2400) \times (60 / 0,01) = 100$$

KODDA SIK YAPILAN HATA

Sayacı iki kez okumak yerine bir kez okuyup değişkende saklamak gerekir; iki okuma arasında yeni bir kenar gelirse hesaplanan fark tutarsız olur. Sayaç taşmasında doğru sonucu almak için sayaç genişliğine uygun işaretli fark kullanılmalıdır. TIM2 bazı STM32 ailelerinde 32 bittir; reference manual kontrol edilmelidir.

Hall etkisi ve üç bitten altı durum

Hall elemanı, içinden akım geçerken manyetik alana bağlı küçük bir gerilim üretir. Dijital Hall sensöründe bu analog bilgi bir eşik devresiyle 0/1 seviyesine çevrilir; yazılım tarafında temiz bir dijital giriş olur. BLDC motorlarda çoğunlukla üç dijital Hall çıkışı bulunur ve sensörler motor tasarımına göre mekanik ya da elektriksel 120° aralıklı yerleştirilir. Amaç her an rotorun hangi sektörde olduğunu üç bitle kodlamaktır. Hall çıkışı rotorun kaba konumunu verir: sektör bilgisi, açı değil. Açı çözünürlüğü encoder'ın işidir.

Üç dijital giriş teorik olarak $2^3 = 8$ kombinasyon üretir. Normal 120° Hall düzeninde 000 ve 111 fiziksel olarak oluşmaz; bu ikisi geçersiz kabul edilir. Kalan altı kod, dönüş yönüne göre daima aynı halka üzerinde sırayla görünür. Örnek bir ileri sıra: 001 → 101 → 100 → 110 → 010 → 011 → 001. Ters yönde aynı halka ters sırada ilerler. Sensör adlandırması, bit sırası ve motor kablolarına göre sıra farklı görünebilir.

HALL TABLOSU İNTERNETTEN ALINMAZ · HIGH/LOW EVRENSEL DEĞİLDİR

"Kuzey = HIGH, Güney = LOW" evrensel bir kural değildir: sensör tipi (unipolar, bipolar/latching), mıknatıs yönü, ara devre ve çıkış terslemesi sonucu değiştirebilir. Hall kablo sırası, faz kablo sırası ve sensörlerin fiziksel yerleşimi üreticiye göre değişir. Gerçek eşleme motor datasheet'inden, çalışan bir karttan veya düşük enerjili ölçümden çıkarılır; yazılımda sekiz elemanlı bir tabloya konur ve 000 ile 111 "geçersiz" işaretlenir. Geçersiz kod görülmesi arıza işaretidir: kopuk kablo, eksik pull-up, besleme sorunu veya gürültü.

HALL GEÇİŞİNDEN HIZ

$$N_{\text{Hall, mekanik tur}} = 6p \quad \cdot \quad \text{RPM} = (n / 6p) \times (60 / \Delta t) \quad \cdot \quad \text{RPM} = 60 / (6 \cdot p \cdot \Delta t)$$

Bir elektriksel turda 6 sektör, bir mekanik turda p elektriksel tur vardır. $p = 7$ ve iki sektör olayı arası $\Delta t = 1 \text{ ms}$ ise $\text{RPM} = 60 / (6 \cdot 7 \cdot 0.001) \approx 1429$. 4 kutup çiftli bir motorda mekanik turda 24 Hall geçişi görülür; bu yüzden "Hall çözünürlüğü her zaman 6 adım/tur" ifadesi doğru değildir. Tek periyot gürültülü hız üretir, son birkaç geçişin hareketli ortalaması alınır; filtre ağır olursa hız kontrolü geç tepki verir. Düşük hızda sabit pencerede geçiş saymak yerine iki geçiş arasındaki süreyi ölçmek tercih edilir.

Karşılaştırma

ÖZELLİK	ARTIMLI ENCODER	BLDC İÇİNDEKİ 3 DİJİTAL HALL
Ana amaç	Hassas görelî pozisyon ve hız	Elektriksel sektör / komütasyon
Çıkış	A, B; bazen Z	H1, H2, H3

ÖZELLİK	ARTIMLI ENCODER	BLDC İÇİNDEKİ 3 DİJİTAL HALL
Çözünürlük	Genellikle yüksek; modele bağlı	Elektriksel turda 6 sektör
Yön bilgisi	A/B faz sırasından	Hall kod sırasından
Mutlak başlangıç	Tek başına vermez; home/Z gerekebilir	Açılışta sektör bilgisi verir
STM32 çevrebirimi	Timer encoder mode	GPIO + EXTI veya timer Hall interface
Hız ölçümü	Çok uygun	Yapılır; düşük hızda daha kaba
Tipik kullanım	Servo, konum/hız döngüsü	Sensörlü 6-step BLDC sürüş

DÜZELTME · OPEN-LOOP / CLOSED-LOOP

Encoder veya Hall'ın varlığı tek başına sistemi closed-loop yapmaz. Closed-loop, ölçülen değerın hedefle karşılaştırılıp hataya göre komutun düzeltilmesidir. Hall ile yapılan komütasyon geri beslemeli bir anahtarlama işlemidir; hız denetimi ise ancak ölçülen hız hedef hızla karşılaştırılıp PWM buna göre düzeltilirse kapalı çevrim olur.

IMU NEDEN GEREKLİ DEĞİL?

IMU gövdenin doğrusal ivmesini ve açısal hızını ölçer. Six-step'in ihtiyacı ise tek bilgidir: altı elektriksel durumdan hangisi uygulanmalı? Bunu Hall sensörü, encoder, Back-EMF veya sürücünün kendi algılama devresi söyler. IMU bu iş için standart geri besleme elemanı değildir.

BÖLÜM 8

Hall yazılımı: union, bit alanı, maske ve EXTI

Projede Hall verisi HALL_RAW türüyle temsil edilir. Bu yapı aynı 8 bitlik veriye iki farklı görünümünden erişmeyi sağlar: bit bit ve bütün byte olarak.

```
typedef union
{
    struct {
        uint8_t hall_c    : 1;    // bit 0
        uint8_t hall_b    : 1;    // bit 1
        uint8_t hall_a    : 1;    // bit 2
        uint8_t reserved  : 5;    // bit 3..7
    } bits;
    uint8_t val;                // 8 bitin tamamı
} HALL_RAW;
```

Union, üyeleri için ayrı ayrı bellek ayırmaz; bütün üyeler aynı adresi paylaşır. Bu yüzden `bits` ve `val` iki farklı değişken değil, aynı 8 bitlik verinin iki farklı yorumudur. Kod kimi yerde tek sensöre bakmak, kimi yerde bütün kodu tablo girişi olarak kullanmak ister: `val` tablo/switch için, `bits` okunabilirlik için. `raw.val = 0x05` yazılırsa ikilik karşılığı 0000 0101 olur; bit alanlarından okunduğunda `hall_a = 1`, `hall_b = 0`, `hall_c = 1` görülür, yani Hall kodu ABC = 101 olarak yorumlanır. Bit sırası bildirim sırasıyla aynıdır: `hall_c` en düşük bit.

PCo, PC1 ve PC2'yi tek okumayla almak

Bağlantı PCo = Hall C, PC1 = Hall B, PC2 = Hall A ise GPIOC giriş veri register'ının alt üç biti union düzeniyle birebir uyuşur. Üç pini ayrı ayrı okumak yerine register bir defa örneklenir; üç sensör aynı anda yakalanmış olur. Üç ayrı okuma arasında rotor dönmeye devam eder ve sektör sınırında karışık bir kod okunabilir.

```
static HALL_RAW hall_oku(void)
{
    HALL_RAW raw = {0};
    // IDR'nin yalnızca bit 0, 1 ve 2'sini al.
    raw.val = (uint8_t)(GPIOC->IDR & 0x07U);
}
```

```
    return raw;
}
```

0x07U maskesi ikilik olarak 0000 0111'dir. AND işlemi PC0–PC2 bitlerini korur, diğer pinlere ait bitleri sıfırlar. Böylece reserved alanı daima 0 kalır ve val doğrudan tablo indeksi olarak kullanılabilir. Maskeleme olmadan val diğer pinlerin durumunu da taşır ve tablo indeksi bozulur.

EXTI kesmesinde kullanım

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0 || GPIO_Pin == GPIO_PIN_1 ||
        GPIO_Pin == GPIO_PIN_2) {
        HALL_RAW hall = hall_oku();
        if (hall.val == 0x00U || hall.val == 0x07U) {
            mosfetleri_kapat();    // 000 ve 111 gecersiz
            return;
        }
        komutasyon_uygula(hall.val);
    }
}
```

BIT-FIELD TAŞINABİLİRLİĞİ

C standardı, bit-field üyelerinin byte içindeki yerleşim sırasını tamamen sabitlemez; düzen derleyici ve hedef mimariye bağlı olabilir. Kullanılan ARM GCC/Clang ile düzen doğrulanmalı ve boyut kontrolü eklenmelidir. Donanım register'larında en taşınabilir yöntem maske ve kaydırmadır: `(raw.val >> 2) & 1U`. Union görünümü okunabilirlik sağlar, maskeler bit konumunu açıkça garanti eder.

```
_Static_assert(sizeof(HALL_RAW) == 1U, "HALL_RAW tam olarak 1 byte olmalı");

#define HALL_C_MASK (1U << 0)
#define HALL_B_MASK (1U << 1)
#define HALL_A_MASK (1U << 2)
#define HALL_MASK   (HALL_A_MASK | HALL_B_MASK | HALL_C_MASK)
```

HIZLI TEKRAR · BÖLÜM 8

1. `raw.val = 0x06` yazılırsa `hall_a`, `hall_b` ve `hall_c` ne olur?
2. Neden üç ayrı `ReadPin` çağrısı yerine tek register okuması tercih edilir?
3. `& 0x07U` maskesi kaldırılırsa hangi hata ortaya çıkar?

BÖLÜM 9

Güç katı: yarım köprü, inverter, gate driver, dead-time

Tüm güç katı tek bir yapının üç kopyasıdır: yarım köprü. Önce onu tam anlayın, inverter kendiliğinden anlaşılır. Bir fazı DC+ veya GND'ye bağlamak için iki anahtar vardır: high-side ve low-side. Dört kombinasyonun üçü kullanılır, biri yasaktır.

ÜST AÇIK

Faz DC+ tarafına çekilir, akımın kaynağı olur.

ALT AÇIK

Faz GND'ye çekilir ve akımın dönüş yolu olur.

İKİSİ KAPALI

Faz boşta (yüksek empedans). Back-EMF tam bu anda ölçülür.

YASAK

Shoot-through: DC+ doğrudan GND'ye bağlanır. Mikrosaniyeler bile sürücüyü bozabilir.

DC+ · DC-link —QUH— —QVH— —QWH— | U | V | W ● = faz orta noktası → motor sargı ucu —QUL— —QVL—
—QWL— GND BLDC MOTOR · üç sargı U-V-W

Gate driver ne yapar, motor driver'dan farkı ne?

GATE DRIVER'IN DÖRT GÖREVİ

- 1 · MOSFET kapılarını yeterli akımla hızlı açıp kapatmak
- 2 · High-side MOSFET için gereken yüksek kapı gerilimini üretmek
- 3 · Bazı modellerde dead-time, düşük gerilim, aşırı akım ve sıcaklık koruması
- 4 · Fault bilgisini mikrodeneleyiciye bildirmek

İKİSİNİ KARIŞTIRMAYIN

Gate driver yalnız kapıları sürer; güç MOSFET'leri harici olabilir, yani güç kartı da sizin işiniz olur. Motor driver kartı gate driver, MOSFET'ler, akım ölçümü ve korumayı birlikte barındırabilir. Kart şeması görülmeden bu iki terim birbirinin yerine kullanılmaz.

Dead-time: yasak durumu imkânsız kılmak

MOSFET anında kapanmaz. Üst anahtara "kapan" dedikten sonra gerçekten kapanması zaman alır, bu yüzden alt anahtar hemen açılmaz. Araya kasıtlı bir boşluk konur; bu sürede faz boşta ve akım diyottan devam eder. Dead-time'ı hem timer hem driver ekliyorsa toplam süre beklediğinizden uzun olur: tork düşer, akım dalgalanır. Dead-time tahminle seçilmez; MOSFET ve driver gecikmeleri, propagation delay farkı ve üretici önerisiyle belirlenir.

HIGH ve LOW, mikrodnetleyici pininin basitçe 1/0 yapılması demek değildir; üç fazlı inverterde ilgili üst veya alt MOSFET'in kontrollü sürülmesini temsil eder. Aynı bacağıın üst ve alt MOSFET'i aynı anda açılırsa shoot-through oluşur ve güç katı anında zarar görür. Gate driver ve dead-time kullanılmadan güç katı sürülmemelidir.

BÖLÜM 10

Sensorless: Back-EMF, zero crossing ve başlatma

Hangi geri beslemeyi kullanacağınız motorun kablo sayısına bakılarak anlaşılır. **3 kalın kablo + ince sensör kabloları:** Hall tabanlı six-step, kolay yol. Rotorun sektörü dururken bile bilinir, motor doğru adımdan başlar. **Yalnızca 3 kalın U-V-W kablosu:** sensorless Back-EMF veya sürücü içi algılama, zor yol. Konum ölçümle tahmin edilir; motor dururken bu bilgi yoktur, bu yüzden ayrı bir başlatma dizisi gerekir.

Hall yoksa rotorun yerini motorun kendisi söyler: mıknatıslar dönerken sargılarda gerilim indükler, buna Back-EMF denir. Six-step'in güzelliği şudur — her adımda bir faz zaten boşta, yani ölçüme hazırdır. Dört kare: (1) iki faz akım taşıırken üçüncünün iki anahtarı da kapalıdır, gerilimi serbestçe okunur. (2) Rotor döndükçe boş fazda yamuk (trapezoidal) bir gerilim belirir; hız arttıkça genlik artar. (3) Sinyal referans eşiğini geçtiği an yakalanır: bu *zero crossing*'dir. (4) Zero crossing komütasyon anı değildir; yaklaşık 30 elektriksel derece beklenir, sonra geçilir. Anahtarlama sonrasındaki güvenilir demanyetizasyon bölgesi maskelenir.

Sensorless motor dururken kendini neden bulamaz?

Back-EMF genliği hızla artar; motor duruyorsa ölçülecek bir şey yoktur. Bu yüzden sensorless başlatma dört ayrı aşamadır.

AŞAMA	YAZILIM NE YAPAR
1 • IDLE	Motor duruyor, konum bilinmiyor. Duty sıfır, çıkışlar güvenli kapalı.
2 • ALIGN	Sınırlı akımla bilinen bir durum uygulanır; rotor bilinen konuma çekilir.
3 • OPEN LOOP	Geri besleme olmadan, hesaplanmış zamanlarla adımlar ilerletilir. Motor hızlanır.
4 • CLOSED LOOP	Back-EMF güvenilir olunca zero crossing takibine geçilir. Olmazsa STARTUP FAULT: güç güvenli kesilir.

Rampa çok hızlıysa rotor alanı takip edemez ve senkron kaybolur; çok yavaş veya duty çok düşükse yük aşılamaz; duty çok yüksekse aşırı akım ve mekanik sıçrama olur. Bu yüzden başlangıç parametreleri akım sınırı altında ayarlanır.

Bu bölümün sonucu: ek Hall kabloları varsa Hall tabanlı six-step daha doğrudan başlar. Yalnız U-V-W varsa Back-EMF algılama veya sürücünün kendi sensorless özelliği araştırılır. Hangi yolun kullanılacağı motor ve sürücü modeli görülmeden kesinleştirilemez.

BÖLÜM 11

Motor kontrol timer'ı, ADC ve capture

Six-step yazılımının kalbi ana döngü değil, timer'dır. Bu bölümdeki her özellik tek bir amaca hizmet eder: altı çıkışın doğru anda, birlikte ve güvenli değişmesi.

BAŞLIK	STM32 YAKLAŞIMI	ESP32 YAKLAŞIMI
PWM birimi	TIM1/TIM8 gibi advanced-control timer'lar	MCPWM timer/operator/generator
Complementary PWM	CHx ve CHxN çıkışları	İki generator ve dead-time modülü
Hall ölçümü	Timer Hall/encoder modu veya input capture / EXTI	MCPWM capture veya GPIO interrupt
Sensorless ölçüm	Timer tetiklemeli ADC, analog watchdog / comparator	ADC/comparator imkânı modele bağlı
Fault	BKIN/BRK ile donanımsal timer kapatma	MCPWM fault/brake işlemleri
Hazır motor araçları	X-CUBE-MCSDK ve Motor Control Workbench	ESP-IDF örnekleri ve özel kod

Karar "hangisi daha güçlü" sorusuyla değil, üç maddeyle verilir: (1) verilen sürücü kartıyla pin ve gerilim uyumu, (2) ekibin yazılım ekosistemi, (3) gereken timer/ADC/capture/fault özelliklerinin seçilen tam modelde bulunması.

Timer'da öğrenilecek dört özellik

A · TEK ZAMAN TABANI

U, V ve W aynı timer'dan üretilir. Ayrı, senkron olmayan timer'lar geçiş anlarını belirsizleştirir. STM32'de tipik eşleme TIM1_CH1/CH1N = U kolu, CH2/CH2N = V, CH3/CH3N = W (gerçek pinler alternate function tablosundan seçilir).

B · EDGE vs CENTER ALIGNED

Edge-aligned'da sayaç tek yönde ilerler, darbe periyodun başına yapışır. Center-aligned'da sayaç yukarı-aşağı sayar, darbe ortalanır; akım ve Back-EMF ölçümü için sakın noktalar daha düzenli oluşur.

C · PRELOAD VE COM

D · BREAK / FAULT GİRİŞİ

Duty veya komütasyon deseni periyodun ortasında değişirse kısa ya da uzamış darbe çıkar. Preload yeni değerleri gölge register'a yazar, tanımlı olayda (update / COM) hepsini birlikte uygular. Altı çıkışın atomik değişmesi böyle sağlanır.

shunt akımı → yükselteç → comparator → TIM BKIN / MCPWM fault → PWM donanımsal kapanır. Aşırı akım fault'u yazılım interrupt'ını beklemez; çıkışlar donanımda güvenli seviyeye çekilir, CPU sonra sebebi kaydeder.

Register isimleri: PSC prescaler, ARR periyot sınırı, CCR1/2/3 kanal karşılaştırma (duty) değerleri. Ayrıntı seriye göre değişir; reference manual esastır.

ADC: ölçümü doğru ana yerleştirmek

Ölçüm hatasının çoğu yanlış ADC ayarından değil, yanlış zamandan gelir. MOSFET anahtarlararken faz ve shunt sinyalleri yüksek frekanslı gürültüyle dolar. Ana döngüde rastgele okumak, çoğu zaman gürültünün tepesini örneklemek demektir. Doğru akış: timer periyodu başlar → geçişten sonra sinyalin oturması için kısa ve bilinen bir süre beklenir → compare olayı ADC'yi tetikler → tamamlanma olayı ölçümü hızlı kontrol döngüsüne verir → yazılım ölçek, offset ve sınır kontrolünü uygular. Ölçüm her periyotta aynı, tekrarlanabilir noktada alınır.

SAYISAL ÖRNEK · SHUNT AKIMI

$$V_{ADC} = ADC_{raw} / (2^N - 1) \cdot V_{ref}$$

$$I = (V_{ADC} - V_{offset}) / (G \cdot R_{shunt})$$

12-bit ADC, $V_{ref} = 3.3$ V, okunan değer 2300 ise $V_{ADC} \approx 1.852$ V. $V_{offset} = 1.65$ V, kazanç $G = 20$, $R_{shunt} = 0.01 \Omega$ için $I = (1.852 - 1.65) / (20 \cdot 0.01) \approx 1.01$ A.

Gerçek tasarımda tolerans, offset kayması, ADC referansı, doyunluk ve ortak-mod sınırları hesaba katılır.

Capture ve sayaç taşması

Hall geçişleri veya Back-EMF olayları arasındaki süre timer capture değeriyle ölçülür. Tek tuzak sayaç sınırına gelip başa dönmesidir. İki olay aynı sayım turundaysa fark doğrudan alınır: $\Delta = \text{yeni} - \text{eski}$. Yeni değer eskiden küçükse fark negatif görünür; doğru süre (periyot - eski) + yeni + 1'dir. Bu kontrol yapılmazsa RPM aniden sıçrar. Interrupt içinde uzun işlem yapılmaz; olay zamanı kaydedilir, gerekli kısa komütasyon işi yapılır, ağır telemetri ana döngüye bırakılır.

BÖLÜM 12

Yazılım mimarisi, durum makinesi ve PI

Motor kontrol yazılımında işler farklı hızlarda koşar. Hepsini tek bir sonsuz döngüye koymak, en hızlı işi en yavaş işin insafına bırakmak demektir — motorun senkron kaybetmesinin en yaygın yazılım sebebi budur.

BEŞ FARKLI ZAMAN ÖLÇEĞİ

PWM ve hızlı ölçüm	mikrosaniye · her periyot
Komütasyon	olay tabanlı · hıza bağlı
Hız kontrolü (PI)	sabit periyot · milisaniye
Telemetri ve komut	yavaş · ana döngü
Fault	anında · en yüksek öncelik

KATMANLAR

Uygulama	start/stop, hedef RPM, yön
Durum makinesi	hangi aşamadayız, fault var mı
Kontrol	rampa ve hız PI
Komütasyon	six-step tablosu · donanımdan bağımsız
Donanım	timer, ADC, GPIO · sürücüye özel eşleme

Her katman altındakini kullanır, üstündekini bilmez. Komütasyon tablosu saf faz durumu üretir; onu gerçek timer çıkışlarına çevirmek yalnız donanım katmanının işidir. Bu ayrım sayesinde tablo ve durum makinesi bilgisayarda test edilebilir.

Motor durum makinesi

INIT → IDLE → CALIBRATION → STARTUP → RUN → STOPPING ↓ start · geri besleme hazır · stop FAULT ← aktif durumların hepsi hata anında buraya geçer FAULT çıkışı tek yönlüdür: hata temizlenir, şartlar sağlanırsa IDLE'a dönülür.

DURUM	O DURUMDA YAPILAN İŞLER
INIT	Clock, GPIO, timer, ADC ve sürücü arayüzü hazırlanır. PWM çıkışları güvenli kapalıdır.
IDLE	Motor enerjisizdir, duty sıfırdır. Komut beklenir.
CALIBRATION	Akım sensörü offset'i, bus gerilimi ve girişlerin geçerliliği kontrol edilir.
STARTUP	Hall varsa mevcut koddan ilk sektör seçilir; sensorless ise hizalama ve açık çevrim hızlandırma yapılır.

DURUM	O DURUMDA YAPILAN İŞLER
RUN	Komütasyon geri beslemeyle sürdürülür; hız kontrolü ve sınırlar çalışır.
STOPPING	Duty kontrollü azaltılır veya uygulamanın gerektirdiği frenleme uygulanır.
FAULT	PWM donanımsal kapatılır, fault sebebi kaydedilir, güvenli yeniden başlatma şartı beklenir.

Hız kontrolü: P ve I'yı sezgisel anlamak

Hata, hedef RPM ile ölçülen RPM'in farkıdır. **Sadece P, küçük K_p :** yavaş yaklaşır ve hedefe tam oturmaz, kalıcı hata kalır. **Sadece P, çok büyük K_p :** sert tepki verir, hedefi aşar, salınır; motorda ses ve akım dalgalanması duyulur. **P + I dengeli:** I terimi uzun süre kalan hatayı biriktirip kapatır. İlk motor döndürmede PI zorunlu değildir: önce sabit ve sınırlı duty ile doğru komütasyon doğrulanır, RPM ölçümü güvenilir olduktan sonra kapalı çevrim eklenir.

$$e = \text{RPM}_{\text{hedef}} - \text{RPM}_{\text{ölçülen}} \quad \cdot \quad u = K_p \cdot e + K_i \cdot \int e \, dt$$

Hedef 1500, ölçülen 1400 RPM ise $e = 100$. $K_p = 0.0002$ (duty/RPM) için P katkısı 0.02, yani duty +%2. Çıkış her zaman sınırlanır: $D_{\min} \leq D \leq D_{\max}$. Duty sınırı dayandığında integral biriktirmeyi durdurun (anti-windup); aksi halde hata kapandıktan sonra motor hedefi uzun süre aşar.

INTERRUPT İÇİNDE YAPILMAYACAKLAR

HAL_Delay veya blocking bekleme · belirsiz süreli döngüler · uzun printf, blocking UART aktarımı · dinamik bellek ayırma · fault çıkışını kapatıp uzun hesap yaptıktan sonra PWM durdurmak · öncelik sırası belirsiz iç içe kesme kurgusu. ISR olayı zaman damgasıyla kaydeder, kısa ve deterministik işi yapar, gerisini bayrak veya kuyrukla ana göreve bırakır.

BÖLÜM 13

Uygulama A · NUCLEO-G491RE üzerinde komütasyon simülasyonu

Bu uygulama gerçek bir BLDC motoru döndürmez. Üç GPIO pini üzerinde BLDC'nin altı adımlı komütasyon sırasını canlandırır. Her 200 ms sonunda A, B ve C fazının durumu değişir; aynı bilgi LPUART1 ile bilgisayara gönderilir ve CoolTerm'de metin veya grafik olarak izlenir. Böylece Bölüm 6'daki tablo, güç katı olmadan gözle takip edilebilir hâle gelir.

NEDEN GERÇEK MOTOR DEĞİL?

PA4, PB4 ve PB5 yalnızca 3.3 V mantık çıkışıdır. Motor fazı veya MOSFET kapısı bu pinlere doğrudan bağlanmaz. Gerçek sistemde 3-faz inverter, gate driver, akım ölçümü ve PWM gerekir. Bu çalışma yalnızca komütasyon mantığını güvenli biçimde öğretir.

Komütasyon tablosu → GPIO A/B/C → LPUART1 TX → ST-LINK Virtual COM Port → CoolTerm

13.1 Nucleo kartı işi neden kolaylaştırır?

Kart üzerindeki ST-LINK V3E iki ayrı iş yapar: SWD ile firmware yazma ve hata ayıklama, Virtual COM Port (VCP) ile UART verisini USB üzerinden bilgisayara taşıma. Bu yüzden ek USB–UART dönüştürücü ve DFU modu gerekmez.

Mac + CoolTerm | micro-USB veri kablosu ▼ NUCLEO üzerindeki ST-LINK V3E | SWD ↔ PA13 (SWDIO), PA14 (SWCLK) : programlama / debug └ VCP ↔ PA2 (LPUART1_TX), PA3 (LPUART1_RX) : UART ▼ STM32G491RET6

Board Selector ile kart seçilmesinin sebebi: CubeMX kart şablonunda ST-LINK ve VCP bağlantılarını (PA2/PA3) tanır. Yalnız MCU seçilirse bu fiziksel bağlantıları kendiniz doğrulamanız gerekir.

13.2 CubeMX ayarları

SYS VE LPUART1

Debug	Serial Wire · PA13/PA14 ayrılır
Timebase	SysTick

ÜÇ FAZ GPIO ÇIKIŞI

PA4	PHASE_A	Output PP, Low, No pull, Low speed
PB4	PHASE_B	Output PP, Low, No pull, Low speed

Mode	Asynchronous
Baud	115200 · 8 Bits / None / 1
Flow control	Disable
Pinler	PA2 = TX, PA3 = RX

PB5	PHASE_C	Output PP, Low, No pull, Low speed
-----	---------	------------------------------------

Bu üç pin projede boş olan bağımsız GPIO'lardır. PA5 karttaki LD2 kullanıcı LED'i olduğu için faz pini seçilmedi. Başlangıcın Low olması reset sonrası güvenli durum verir.

LPUART alanı gri veya kilitli görünürse kart şablonu bazı pinleri önceden ayırmış olabilir; Pinout menüsünden Clear Pinout / Reset Configuration ile temizlenip ayarlar yeniden yapılır. Mevcut projede rastgele yapılmaz, önceki pin ayarlarını siler. Project Manager'da Keep User Code açık bırakılır: .ioc değişip tekrar Generate Code yapıldığında USER CODE BEGIN/END bloklarındaki uygulama kodu korunur.

13.3 Kodun mantığı

Faz durumu üç değerli bir tiptir. FLOAT ile LOW aynı şey değildir: LOW aktif olarak GND'ye çeker, FLOAT hiç sürmez. Gerçek motor sürücüsünde FLOAT, ilgili faz kolunun tamamen kapatılmasına karşılık gelir.

```
typedef uint8_t PhaseState_t;
enum { PHASE_FLOAT = 0U, PHASE_LOW = 1U, PHASE_HIGH = 2U };

#define MOTOR_STEP_COUNT      6U
#define MOTOR_STEP_PERIOD_MS 200U
#define PACK_PHASES(a, b, c)  (uint8_t)(((a) << 4) | ((b) << 2) | (c))
#define UNPACK_PHASE(v, shift) (PhaseState_t)((v) >> (shift)) & 0x03U

static const uint8_t commutation_table[MOTOR_STEP_COUNT] = {
    PACK_PHASES(PHASE_HIGH, PHASE_LOW, PHASE_FLOAT), /* 1 */
    PACK_PHASES(PHASE_HIGH, PHASE_FLOAT, PHASE_LOW), /* 2 */
    PACK_PHASES(PHASE_FLOAT, PHASE_HIGH, PHASE_LOW), /* 3 */
    PACK_PHASES(PHASE_LOW, PHASE_HIGH, PHASE_FLOAT), /* 4 */
    PACK_PHASES(PHASE_LOW, PHASE_FLOAT, PHASE_HIGH), /* 5 */
    PACK_PHASES(PHASE_FLOAT, PHASE_LOW, PHASE_HIGH) /* 6 */
};
```

PACK_PHASES üç faz durumunu ikişer bit hâlinde tek byte'ta saklar (3 durum için 2 bit yeter): A 4–5. bite, B 2–3. bite, C 0–1. bite yerleşir, tablo altı byte tutar. const olduğu için Flash'a yerleşir, RAM harcamaz. FLOAT'ın 0 olması ayrıca bir güvenlik tercihidir: sıfırlanmış bir değer, sürülmeyen faz demektir.

```
static void Set_Phase(GPIO_TypeDef *port, uint16_t pin, PhaseState_t state)
{
    GPIO_InitTypeDef gpio = {0};      /* kullanılmayan alanlar sıfırlanır */
    gpio.Pin    = pin;
    gpio.Pull   = GPIO_NOPULL;
    gpio.Speed  = GPIO_SPEED_FREQ_LOW;

    if (state == PHASE_FLOAT) {
        gpio.Mode = GPIO_MODE_INPUT;   /* surme yok */
        HAL_GPIO_Init(port, &gpio);
    } else {
        gpio.Mode = GPIO_MODE_OUTPUT_PP;
        HAL_GPIO_Init(port, &gpio);
        HAL_GPIO_WritePin(port, pin,
            (state == PHASE_HIGH) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    }
}
```

Her çağrıda HAL_GPIO_Init'in tekrar çağrılma sebebi mod değişimidir: FLOAT için pin Input'a, HIGH/LOW için Output'a çevrilmelidir. Sadece WritePin ile FLOAT üretilemez.

```
static void Motor_Advance(void)
{
    const uint8_t packed = commutation_table[current_step];
    const PhaseState_t a = UNPACK_PHASE(packed, 4);
    const PhaseState_t b = UNPACK_PHASE(packed, 2);
    const PhaseState_t c = UNPACK_PHASE(packed, 0);

    Set_Phase(PHASE_A_GPIO_Port, PHASE_A_Pin, a);   /* önce pinler: us */
    Set_Phase(PHASE_B_GPIO_Port, PHASE_B_Pin, b);
    Set_Phase(PHASE_C_GPIO_Port, PHASE_C_Pin, c);

    char line[48];                                  /* sonra UART: ms */
    const int n = snprintf(line, sizeof line, "STEP %u | A: %d | B: %d | C: %d\r\n",
        (unsigned)(current_step + 1U), Phase_To_Int(a), Phase_To_Int(b), Phase_To_Int(c));
    HAL_UART_Transmit(&hlpuart1, (uint8_t *)line, (uint16_t)n, 100U);
}
```



```
current_step = (uint8_t)((current_step + 1U) % MOTOR_STEP_COUNT);
}
```

Pin güncellemesi mikrosaniyeler, UART milisaniyeler sürer; bu yüzden fazlar önce değişir, satır "şu an olan"ı bildirir. Yaklaşık 30 byte 115200 baud'da 2.6 ms sürer, 200 ms periyoda göre ihmal edilir. Phase_To_Int dönüşümü HIGH → +1, LOW → -1, FLOAT → 0'dır; CoolTerm chart'ının -1/0/+1 görmesinin kaynağı budur.

```
int main(void)
{
    HAL_Init();                /* SysTick 1 ms, NVIC, Flash cache */
    SystemClock_Config();      /* HSI + PLL -> 170 MHz */
    MX_GPIO_Init();            /* PA4/PB4/PB5 Output PP Low */
    MX_LPUART1_UART_Init();    /* 115200 8-N-1 */

    Send_Chart_Header();        /* CoolTerm için açıklama satırları */
    uint32_t last_step_ms = HAL_GetTick();
    Motor_Advance();            /* beklemeden STEP 1'i uygula */

    while (1) {
        const uint32_t now_ms = HAL_GetTick();
        if ((uint32_t)(now_ms - last_step_ms) >= MOTOR_STEP_PERIOD_MS) {
            last_step_ms += MOTOR_STEP_PERIOD_MS;
            Motor_Advance();
        }
    }
}
```

NEDEN HAL_DELAY DEĞİL

HAL_Delay işlemciyi bloklar. HAL_GetTick ile fark almak ana döngüyü serbest bırakır; ileride buton okuma veya UART RX eklemek kolaylaşır. Fark unsigned alındığı için HAL_GetTick 49.7 günde taşsa bile sonuç doğru çıkar.

NEDEN += PERIOD

= now_ms yazılsaydı döngünün gecikmesi her adımda birikirdi. Toplama ile ortalama periyot tam 200 ms kalır. Bir tam tur $6 \times 200 \text{ ms} = 1.2 \text{ s}$ sürer.

13.4 Derleme, yükleme ve CoolTerm

CubeIDE'de çekiç simgesiyle derlenir; beklenen çıktı 0 errors, 0 warnings ve Debug/staj_motor_simule_g491.elf. Kart USB ile bağlıyken Run/Debug ST-LINK üzerinden SWD ile yazar ve MCU'yu resetler; DFU moduna gerek yoktur. Build çıktısındaki text 12176 kod ve const tabloları, data 12 ilk değerli değişkenleri, bss 1724 sıfırlanan RAM'i gösterir: toplam 13.9 KB, G491'in 512 KB Flash'ının yüzde üçü. "ST-Link Server is required" uyarısı kod hatası değildir; macOS'ta eksik bileşen kurulur veya .elf, STM32CubeProgrammer ile ST-LINK/SWD üzerinden yazılır.

CoolTerm tarafında port /dev/cu.usbmodem..., baud 115200, 8-N-1, flow control None seçilir. Connect'e basınca STEP 1–6 satırları yaklaşık her 200 ms'de bir görünür. Sondaki port numarası her bağlanışta değişebilir; önemli olan ST-LINK'e ait girdiyi seçmektir. Metin bozuk geliyorsa ilk kontrol baud, sonra 8-N-1 ve flow control'dür; port hiç görünmüyorsa veri taşıyan bir USB kablosu kullanılmalı ve başka bir terminalin portu açık tutmadığından emin olunmalıdır.

```
STEP 1 | A: 1 | B: -1 | C: 0      A: +1, +1, 0, -1, -1, 0, ...
STEP 2 | A: 1 | B: 0 | C: -1     B: -1, 0, +1, +1, 0, -1, ...
STEP 3 | A: 0 | B: 1 | C: -1     C: 0, -1, -1, 0, +1, +1, ...
STEP 4 | A: -1 | B: 1 | C: 0
STEP 5 | A: -1 | B: 0 | C: 1      Fazlar birbirine 120 derece,
STEP 6 | A: 0 | B: -1 | C: 1      yani iki adım kaymıştır.
```

13.5 CoolTerm Chart: dört trace ve yorumlanması

CoolTerm satırdaki sayıları sırayla ayıklar: Trace 1 = STEP numarası, Trace 2/3/4 = A/B/C. 0–6 arasında yükselen ilk eğri faz gerilimi değil, adım numarasıdır. Temiz grafik için View > Chart'a geçilir, sağ tık > Chart Properties'te "Only plot lines with exactly the specified number of data points" açılıp 4 yapılır, Trace Properties'te Trace 1 gizlenir, Trace 2–4 Phase A/B/C olarak adlandırılır, Clear Data ile birkaç saniye yeni veri biriktirilir. Y ekseninin anlamı: +1 faz GPIO'su aktif 3.3 V sürüyor, 0 FLOAT (input / yüksek empedans), -1 aktif 0 V. Eğrilerin sinüs değil basamaklı olması doğru sonuçtur; six-step trapezoidal sinyal üretir.

Hem okunaklı metin hem temiz chart isteniyorsa iki satır türü gönderilir: STEP 1 | A: 1 | B: -1 | C: 0 terminal için, 1, -1, 0 chart için. Chart Properties'te "yalnızca 3 sayı içeren satırları çiz" filtresi seçilir.

13.6 Başarı kontrol listesi

CubeIDE · 0 errors, 0 warnings

ST-LINK · firmware karta yazılabiliyor

CoolTerm · doğru port, 115200 8-N-1, flow control None

Terminal · STEP 1'den 6'ya sıra sürekli tekrar ediyor

Chart · A/B/C yalnızca -1, 0 ve +1 seviyelerinde; her adımda bir HIGH, bir LOW, bir FLOAT

Özet: CubeMX donanımı tanımlar; CubeIDE HAL kodunu derler; ST-LINK firmware'i SWD ile karta yazar; LPUART1 veriyi VCP üzerinden bilgisayara taşır; CoolTerm veriyi terminal ve grafik olarak görünür yapar.

BÖLÜM 14**Uygulama B · STM32F401CCUx ve USB CDC varyantı**

İkinci kart bir Nucleo değil, çıplak bir F401 modülüdür: üzerinde ST-LINK yoktur. Bu iki farkı doğurur. Seri hat, VCP yerine MCU'nun kendi USB birimiyle (USB_OTG_FS, CDC sınıfı) kurulur; firmware SWD yerine USB DFU bootloader'ı ve STM32CubeProgrammer ile yazılır. Komütasyon mantığı, tablo ve Set_Phase yaklaşımı Bölüm 13 ile aynı kalır; değişen yalnızca pin eşlemesi, saat ağacı ve yükleme yoludur.

STM32CubeMX File Window Help

Home > STM32F401CCUx > staj_motor_simule.ioc – Pinout & Configuration > GENERATE CODE

Pinout & Configuration | Clock Configuration | Project Manager | Tools

Software Packs | Pinout

GPIO Mode and Configuration

Mode

Categories A-Z

System Core

- DMA
- ☒ GPIO
- IWDG
- ☒ NVIC
- ☒ RCC
- ☒ SYS
- WWDG

Analog >

Timers >

Connectivity

- I2C1
- I2C2
- I2C3
- SPI1
- SPI2
- SPI3
- USART1
- ⚠ USART2
- 🔴 USART6
- ☒ USB_OTG_FS

Multimedia >

Computing >

Middleware and Software Pac... >

Configuration

Group By Peripherals

☒ GPIO ☒ USB

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin ...	Signal o...	GPIO ou...	GPIO m...	GPIO Pu...	Maximu...	User La...	Modified
PA4	n/a	Low	Output ...	No pull...	Low	PHASE_A	☒
PA5	n/a	Low	Output ...	No pull...	Low	PHASE_B	☒
PA6	n/a	Low	Output ...	No pull...	Low	PHASE_C	☒

🔍 Select Pins from table to configure them. Multiple selection is Allowed.

Pinout view | System view

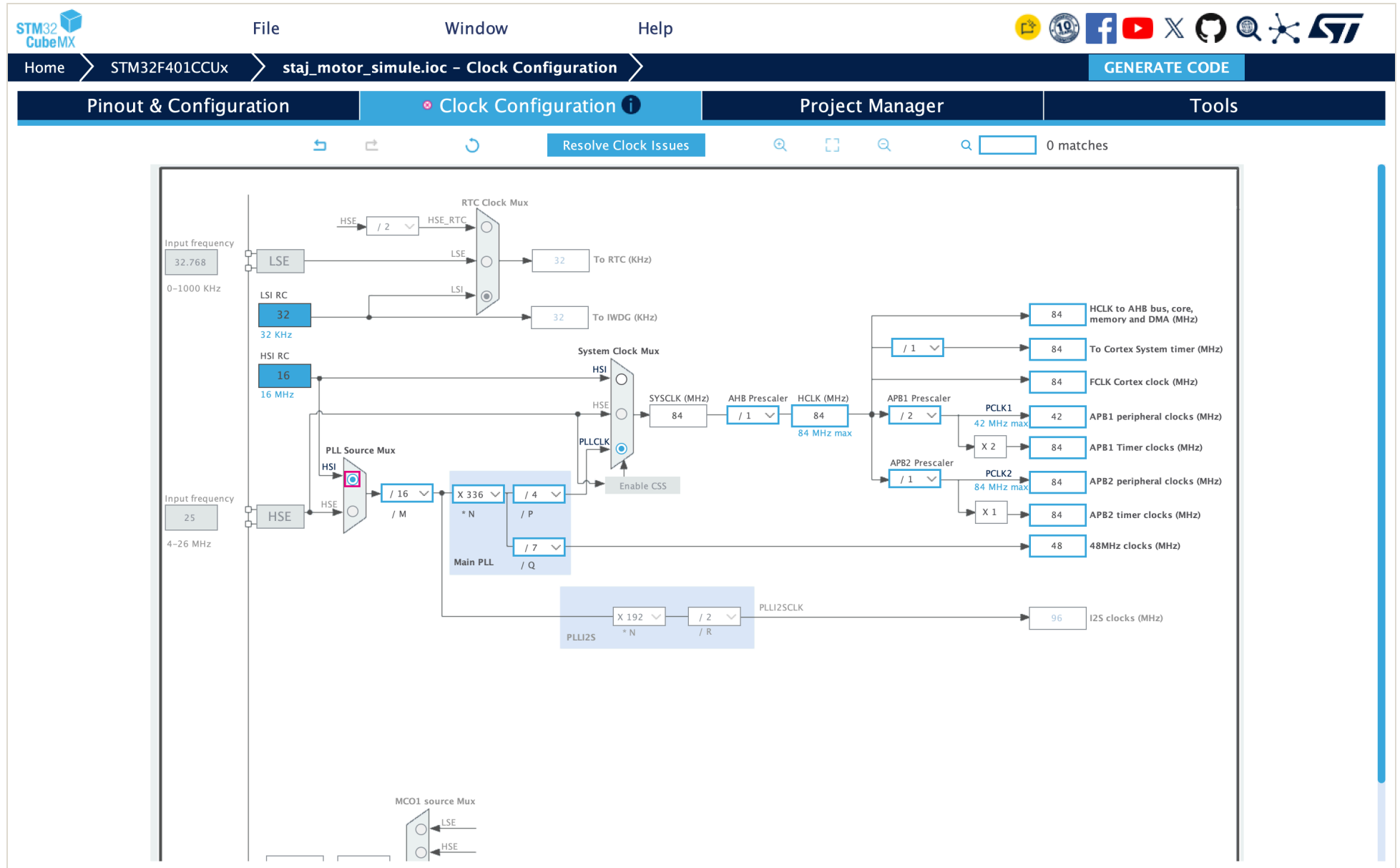
STM32F401CCUx UFPN48

PHASE_A
PHASE_B
PHASE_C

USB_OTG_FS_DF
USB_OTG_FS_DM

Şekil 14.1 · `staj_motor_simule.ioc` · PA4/PA5/PA6 PHASE_A/B/C olarak Output Push Pull, Low, No pull. Sağdaki pinout'ta PA11 ve PA12 USB_OTG_FS_DM / DP için ayrılmış. Connectivity listesinde USB_OTG_FS etkin, USART2 uyarı işaretli.

Faz pinleri bu kartta PA4, PA5 ve PA6'dır — G491'deki PA4/PB4/PB5 ile karıştırılmamalıdır. USB kullanıldığı için PA11 ve PA12 rezerve edilir; bu iki pin başka bir işe verilemez. Middleware tarafında USB_DEVICE sınıfı Communication Device Class (Virtual Port Com) seçilir, böylece bilgisayarda yine bir `/dev/cu.usbmodem...` portu görünür ve CoolTerm hiç değişmeden çalışır. Fark, gönderimin `HAL_UART_Transmit` yerine `CDC_Transmit_FS()` ile yapılmasıdır.

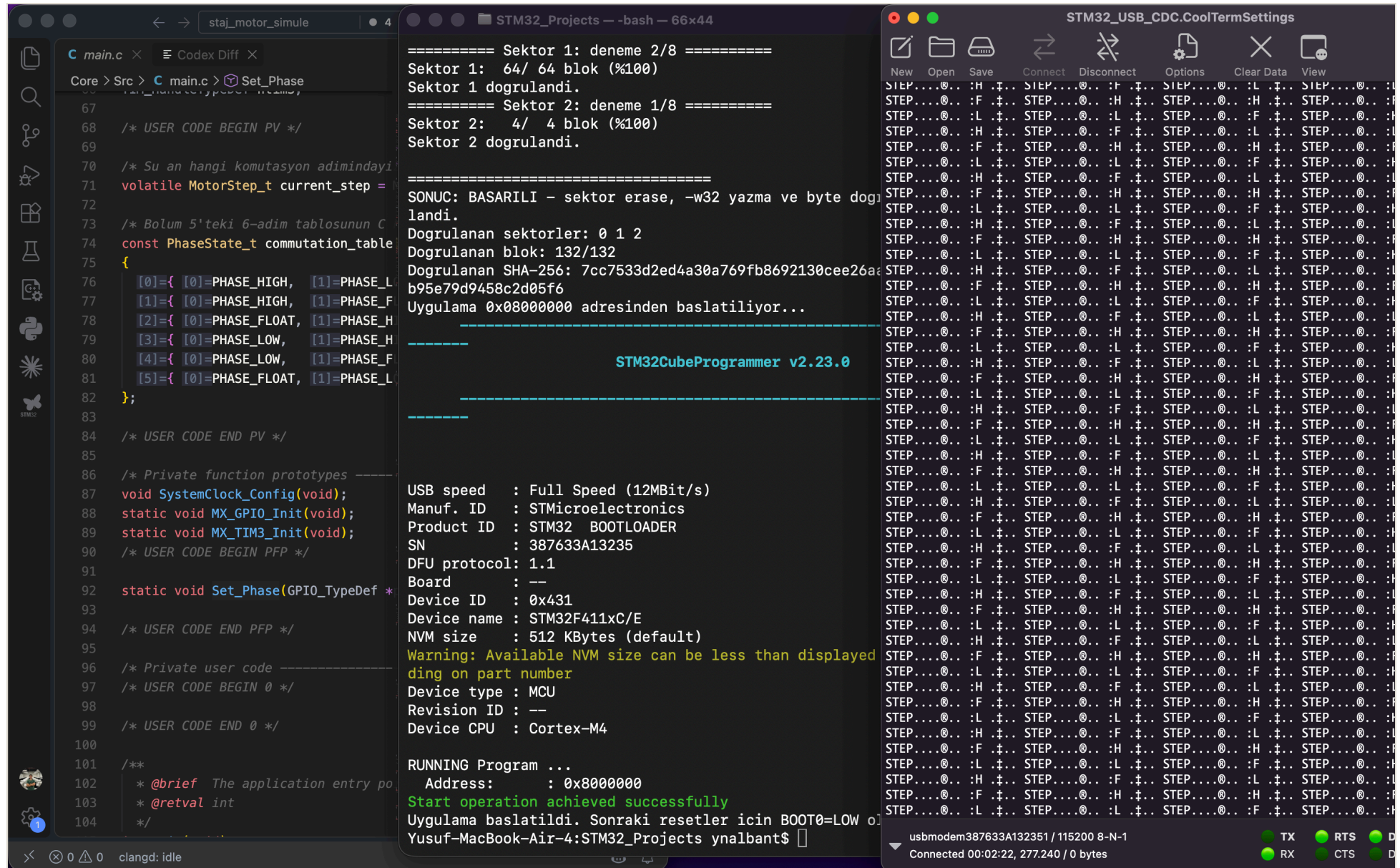


Şekil 14.2 · Saat ağacı: HSI 16 MHz $\rightarrow$ /M = 16 $\rightarrow$ $\times$ N = 336 $\rightarrow$ /P = 4 $\rightarrow$ SYSCLK 84 MHz. /Q = 7 kolu 48 MHz üretir; USB bu kola bağlıdır. APB1 42 MHz, APB2 84 MHz; timer saatleri her iki busta 84 MHz.

USB NEDEN TAM 48 MHZ İSTER?

Full Speed USB 48 MHz'lik bir saat gerektirir ve bu değer yaklaşık olamaz. Q bölücüsü 48 MHz vermezse cihaz bilgisayarda hiç görünmez veya bağlantı sürekli kopar. Bu yüzden F401'de PLL değerleri "84 MHz istiyorum" diye değil, "hem 84 hem tam 48 çıkmalı" diye seçilir. Böyle bir kısıt G491'de LPUART kullanıldığı için yoktu.

Yükleme, kart BOOTo = HIGH ile resetlenip DFU bootloader'a girdikten sonra STM32CubeProgrammer ile yapılır. Programmer sektörleri siler, 32 bitlik yazma yapar ve yazılanı byte byte doğrular; sonunda SHA-256 özeti ile birlikte "BASARILI" raporu verir ve uygulamayı 0x08000000 adresinden başlatır. Sonraki normal açılışlar için BOOTo tekrar LOW konumuna alınır.



Şekil 14.3 · Üç pencere aynı anda: solda main.c içindeki `commutation_table` (PHASE_HIGH / PHASE_LOW / PHASE_FLOAT dizisi) ve `MX_TIM3_Init` prototipi; ortada CubeProgrammer v2.23 çıktısı — sektör doğrulama, SHA-256, Device ID 0x431, Full Speed USB, "Start operation achieved successfully"; sağda CoolTerm penceresine akan STEP satırları, 115200 8-N-1, bağlantı 2 dakikada 277 KB veri.

KONU	NUCLEO-G491RE	STM32F401CCUX MODÜLÜ
Sistem saati	170 MHz (PLLM 4 · N 85 · R 2)	84 MHz (M 16 · N 336 · P 4) + 48 MHz USB
Faz pinleri	PA4, PB4, PB5	PA4, PA5, PA6
Bilgisayara veri	LPUART1 → ST-LINK VCP	USB_OTG_FS · CDC (PA11/PA12)
Gönderim çağrısı	HAL_UART_Transmit	CDC_Transmit_FS
Firmware yazma	SWD · kart üstü ST-LINK V3E	USB DFU · BOOTo + CubeProgrammer
Debug	Doğrudan, breakpoint ile	Harici ST-LINK gerekir
Öğrenme değeri	Hızlı başlangıç, güvenli deneme	USB yığını, bootloader ve saat kısıtları

BÖLÜM 15

Kendi izleme uygulamamız

CoolTerm genel amaçlı bir terminaldir; satırdaki sayıları çizer ama STEP'in ne olduğunu bilmez. Üç eksiği vardır: Trace 1'in adım numarası olduğunu bilmez, her açılışta elle filtre gerekir; tek yönlüdür, karta hız veya başlat/durdur komutu gönderemez; veriyi anlamsız sayı olarak görür, RPM ve adım süresi gibi türetilmiş bilgi üretmez. Motor kontrolüne özel bir uygulama bu üç boşluğu kapatır.

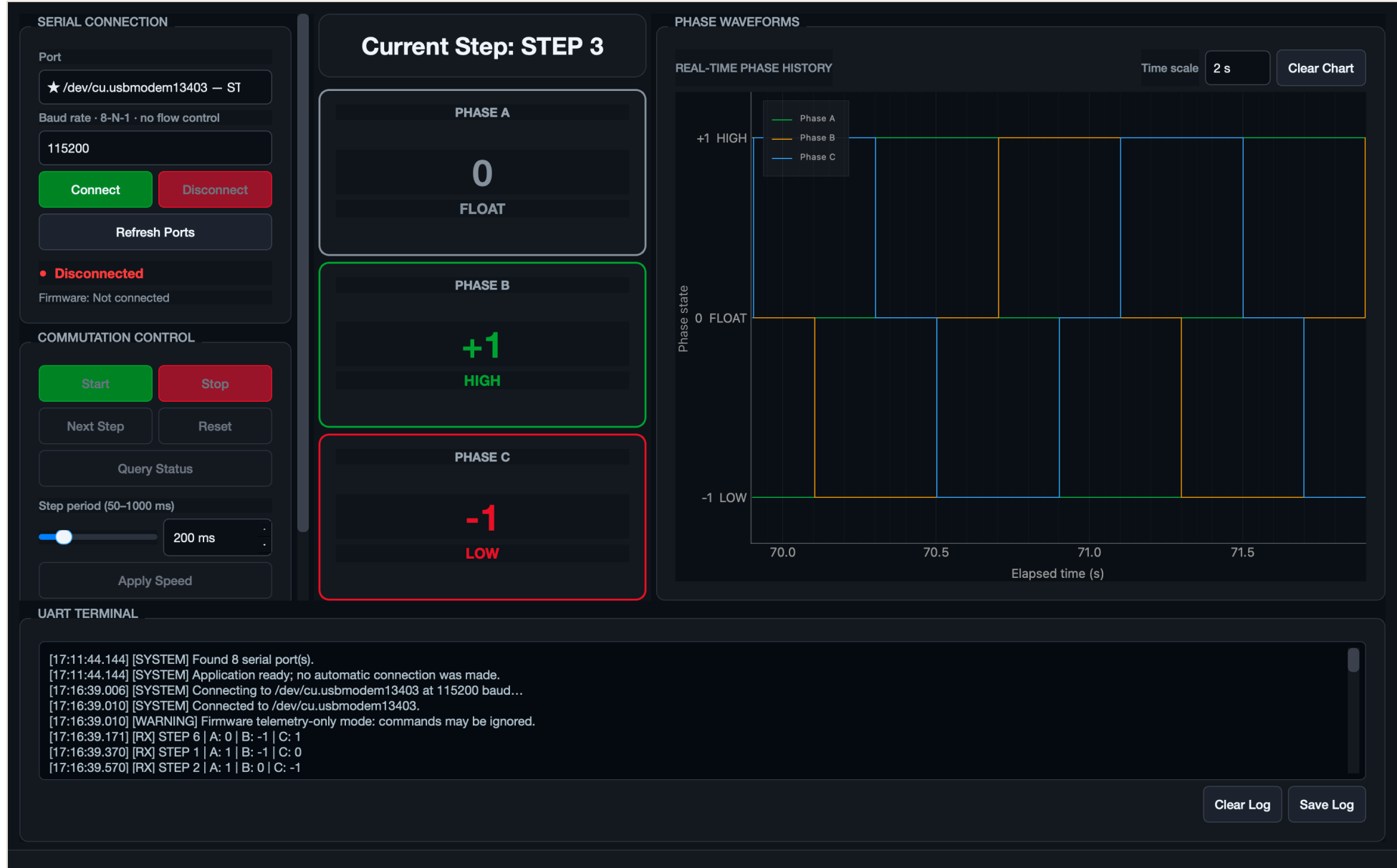
Veri zinciri değişmez, yalnızca son halka yenilenir. Uygulama üç katmandan oluşur ve bağımlılık tek yönlüdür: **seri katman** portu listeler, açar, kapatır, byte okur ve yazar; **veri katmanı** satırı \n'a kadar biriktirir, ayrıştırır, ring buffer'a yazar, periyot ve RPM istatistiğini üretir, "her satırda tam bir HIGH, bir LOW, bir FLOAT var mı" doğrulamasını yapar ve CSV/JSON kaydı tutar; **görünüm katmanı** terminal, faz grafiği, altı dilimli komütasyon dairesi ve panelden oluşur. Veri yukarı akar (byte → satır → kayıt → çizim), komut aşağı iner (düğme → karakter → TX). Katmanlar ayrı tutulursa protokol değiştiğinde yalnızca parser değişir.

HANGI TEKNOLOJİYLE BAŞLAMALI?

Python + pyserial + pyqtgraph: seri okuma on satır, canlı grafik yirmi satır kod. Alternatifler tarayıcıda Web Serial API (kurulum yok, doğrudan usbmodem'e bağlanır), dağıtılabilir bir .app için Electron/Tauri, endüstriyel araç hedefleniyorsa C++/Qt ile QSerialPort ve QtCharts. Protokol oturduktan sonra taşınabilir; parser mantığı aynı kalır.

15.1 Aşama 1 · Mevcut firmware ile

Firmware hiç değişmeden, yalnız STEP n | A: x | B: y | C: z satırıyla ilk sürüm çalışır.



Şekil 15.1 · İlk sürüm. Solda seri bağlantı ve komütasyon kontrolü, ortada A/B/C durum kutuları (0 FLOAT, +1 HIGH, -1 LOW), sağda gerçek zamanlı faz geçmişi 2 s pencerede. Altta zaman damgalı UART terminali; [SYSTEM] ve [RX] satırları ayrı etiketlenmiş, uygulama firmware'in yalnız telemetri gönderdiğini fark edip uyarı yazmış.

Bu aşamada elde edilenler: ham satırların zaman damgalı ve renkli terminali; A/B/C'nin -1/0/+1 basamaklı grafiği ve STEP'in otomatik olarak grafikten ayrılması; altı dilimli komütasyon göstergesi; türetilmiş değerler — adım süresi, elektriksel RPM = $60000 / (6 \times \text{periyot})$, toplam tur ve kaçırılan adım sayısı; her satır için geçerlilik denetimi ve hata sayacı; CSV/JSON kaydı. Adım süresinin ölçülmesi önemlidir: firmware 200 ms der, bilgisayarda ölçülen süre farklıysa ya USB tamponlaması ya ana döngüde bloklama vardır. Aynı ölçüm, ileride timer interrupt'a geçişin etkisini de gösterir.

15.2 Aşama 2 · İki yönlü çalışma ve Hall telemetrisi

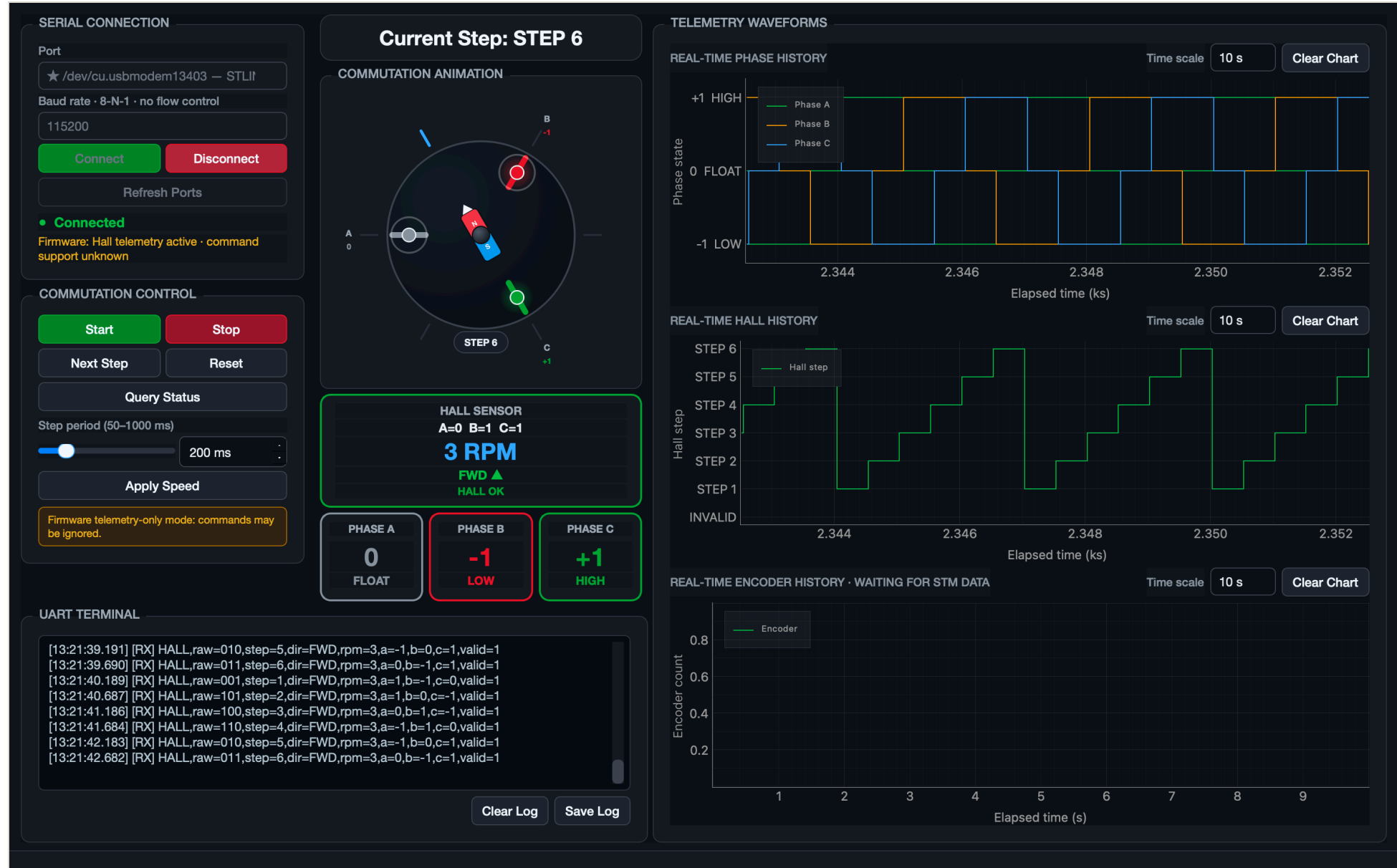
Firmware'de UART RX açılır (PA3 zaten ayarlıdır). HAL_UART_Receive_IT(&hlpuart1, &rx_byte, 1) ile interrupt bazlı tek byte alınır; CubeMX'te LPUART1 global interrupt kutusu işaretlenir. Ana döngü bloklanmaz, komut anında işlenir. Komut seti tek karakterlidir: s start, p pause, + / - periyodu ± 50 ms değiştir, d yön değiştir, n tek adım ilerle, ? durum raporu iste.



Şekil 15.2 · Hall telemetrisi aktif. Ortada komütasyon animasyonu: altı sektörlü daire, rotor mıknatısı ve aktif adım (STEP 2); altında Hall paneli A=1 B=0 C=1, 8 RPM, FWD, HALL OK. Terminal artık yapılandırılmış satır alıyor: `HALL, raw=101, step=2, dir=FWD, rpm=8, a=1, b=0, c=-1, valid=1`. Sarı uyarı, firmware'in komut desteğinin henüz doğrulanmadığını söylüyor.

Satır formatı burada insan okuması için gevşek metinden alan adlı yapıya geçmiştir: `raw` ham üç bitlik Hall kodu, `step` tablodan seçilen adım, `valid` ise 000/111 denetiminin sonucudur. Terminal kaydındaki sıra Bölüm 7'deki halkanın aynısıdır: 001 → 101 → 100 → 110 → 010 → 011 → 001. Uygulama beklenen sırayı bildiği için bir kod atlandığında bunu hata olarak sayabilir.

15.3 Aşama 3 · Encoder kanalı ve büyüyen protokol



Şekil 15.3 · Telemetry bölmesi üç grafiğe ayrıldı: faz geçmiş, Hall adım geçmiş (1–6 arası merdiven ve INVALID satırı) ve encoder sayacı. Encoder başlığı "waiting for STM data" diyor — kanal uygulamada hazır, firmware o alanı henüz göndermiyor. Panel bu durumda boş grafik gösterir, uydurma veri üretmez.

Hall adım geçmişindeki merdiven, sağlıklı komütasyonun en hızlı okunan işaretidir: düzgün 1→6 tırmanışı ve tek basamaklı düşüşler beklenir. Eksen üzerindeki INVALID satırı boşuna değildir; 000 veya 111 geldiğinde eğri oraya düşer ve kablo, pull-up ya da gürültü sorunu anında görünür.

Protokol büyüdüğü üç seçenek vardır. **A · CSV satırı** — `t_ms,step,a,b,c,period_ms`; basittir ve CoolTerm ile de uyumludur. **B · JSON satırı** — `{"t":12400,"step":3,"ph":[0,1,-1],"per":200,"dir":1}`; alan eklemek kolaydır. **C · Binary çerçeve** — `[0xAA][len][type][payload...][crc8]`; PWM ve akım ölçümü eklendiğinde milisaniyede onlarca örnek gelir ve metin 115200 baud'u doldurur, binary hem hız hem hata tespiti sağlar; o noktada baud 921600'e çıkarılabilir. Zaman damgasını kartın göndermesi tercih edilir: bilgisayarın okuma zamanı USB tamponu yüzünden ± 10 ms oynar, HAL_GetTick değeri satıra eklenirse periyot analizi doğru olur.

15.4 Firmware ilerledikçe eklenecek paneller

FIRMWARE ADIMI	KARTTAN GELEN YENİ VERİ	UYGULAMADA GÖRÜNÜM
Timer interrupt	Kesin periyot, jitter	Periyot histogramı; hedef ve ölçülen karşılaştırması
Timer PWM	Duty cycle, PWM frekansı	Duty kaydırıcısı; faz grafiği –duty...+duty aralığında
Hall sensörü	Hall durumu (3 bit), rotor konumu	Dairede beklenen ve ölçülen adım; senkron hatası
Back-EMF	FLOAT fazın ADC gerilimi, sıfır geçiş anı	Osiloskop görünümü; sıfır geçiş işaretleri
Akım ölçümü	Faz akımları, bus akımı	Akım grafiği; aşırı akım alarmı ve eşik çizgisi
Kapalı çevrim hız	Hedef RPM, ölçülen RPM, kontrolcü çıkışı	Hedef/ölçülen üst üste; kazançları canlı ayarlama
Koruma	Hata kodları, sıcaklık, bus gerilimi	Durum ışıkları; hata günlüğü; acil durdur

ACIL DURDUR YAZILIMA BIRAKILMAZ

USB kopabilir. Uygulama p gönderir, ama firmware'de ayrıca watchdog olmalıdır: belirli süre komut gelmezse fazlar FLOAT'a alınır. Donanım koruması — sigorta, akım limiti, donanımsal fault girişi — her zaman yazılımın üstündedir.

15.5 Geliştirme sırası

- 1 · Port listele, bağlan, ham satırları göster. CoolTerm ile birebir aynı davranış: referans noktası.
- 2 · Parser: satırı {step, a, b, c, t} kaydına çevir. Hatalı satırları say, atma.
- 3 · Faz grafiği, komütasyon dairesi ve türetilmiş değerler.
- 4 · CSV kayıt ve tekrar oynatma — kartsız geliştirme mümkün olur, "o an ne oldu" sorusu cevaplanır.
- 5 · Firmware'e RX ekle; s / p / + / - komutları ve hız kaydırıcısı.
- 6 · Protokolü CSV veya JSON'a taşı, zaman damgasını karta al.
- 7 · Firmware ilerledikçe 15.4'teki panelleri tek tek ekle.

BÖLÜM 16

Gerçek motora geçiş: aşamalı plan, hata ayıklama, güvenlik

Buraya kadar komütasyon mantığı GPIO ve seri hat üzerinde doğrulandı. Gerçek motora geçiş bir sonraki tek adım değil, dokuz aşamalı bir sıradır. Her aşamanın bir amacı ve bir geçiş kriteri vardır; kriter sağlanmadan sonraki aşamaya geçilmez. Hataların çoğu atlanan aşamadan gelir.

Aşama 0 · Donanım kimlik kartı

Motoru döndürmeye çalışmadan önce bu kart doldurulur. Boş kalan her satır, sonraki aşamalarda bir hata olarak geri döner.

Motor tam model numarası

Sürücü giriş modu: 3-PWM / 6-PWM / komut

Nominal ve maksimum gerilim

Logic gerilimi ve aktif seviye

Sürekli ve tepe akımı

Dead-time'ı kim ekliyor?

Kutup çifti sayısı (p)

Akım, BEMF ve fault çıkışları

Hall kabloları var mı?

STM32 veya ESP tam modeli

Sürücü kartı ve entegresi

Aşama 1 · Datasheet'ten bağlantı tablosu

İŞLEV	SÜRÜCÜ PINİ	MCU PINİ	KONTROL NOTU
U high / PWM A			Timer alternate function
U low			Complementary çıkış veya sürücü içi
V high / PWM B · V low			Aynı düzen
W high / PWM C · W low			Aynı düzen
Enable			Reset anında güvenli seviye
Fault			BKIN veya MCPWM fault

İŞLEV	SÜRÜCÜ PINİ	MCU PINİ	KONTROL NOTU
Akım ölçümü			ADC kanal ve ölçek
BEMF U/V/W · Hall 1/2/3			ADC/comparator; input capture / EXTI · varsa

Yalnız sinyal bağlantıları. Sürücü girişleri 3.3 V uyumlu olmayabilir veya aktif-low olabilir; GND referansının nasıl bağlandığı, izolasyon olup olmadığı ve maksimum pin gerilimleri datasheet'ten doğrulanır.

Aşama 2–4 · Güç vermeden kanıtlama

2 · PWM ÜRETME

Amaç motoru döndürmek değil, doğru sinyalin üretildiğini kanıtlamaktır. Sürücü enable pasif tutulur; timer tek güvenli frekansta yapılandırılır; üç kanalın aynı periyotta olduğu ölçülür; complementary polarite kontrol edilir; dead-time osiloskopla ölçülür; MCU reset ve debug duraklamasında çıkışların güvenli kaldığı doğrulanır.

Kriter: high ve low hiçbir anda çakışmıyor; frekans ve duty hesaplanan değerde; enable kapalıyken anahtarlar iletme geçmiyor; fault'ta PWM donanımsal kapanıyor.

3 · TABLO TESTİ

Her adım düşük hızda yapay olarak ilerletilir, altı çıkış logic analyzer ile izlenir. Her adımda yalnız doğru faz çifti aktif mi? Boş fazın iki anahtarı da kapalı mı? Adım değişiminde kısa yanlış kombinasyon var mı? İleri ve geri sıra doğru mu? Geçersiz state'te tüm çıkışlar kapanıyor mu?

Kriter: altı adımın tamamı beklenen desende, ara geçişlerde yanlış kombinasyon olmadan gözlenir.

4 · GERİ BESLEME

Hall varsa motor enerjisiz elle yavaşça çevrilir, Hall kodu ve zaman damgası UART'a kaydedilir: altı geçerli kod periyodik tekrar etmeli, 000/111 sürekli görülmemeli, ileri çevirince kodlar bir sırada, geri çevirince ters sırada ilerlemeli.

Sensorless ise BEMF algılama donanımı kartta doğrulanır. Faz gerilimleri uygun direnç bölücülerden geçmelidir; **faz hattı doğrudan ADC pinine bağlanmaz.**

Aşama 5–7 · İlk dönüş, RPM, kapalı çevrim

Motor mekanik olarak sabitlenir, tehlikeli yük veya pervane çıkarılır. Laboratuvar kaynağı düşük gerilim ve düşük akım limitiyle ayarlanır, başlangıç duty'si sınırlı seçilir. Hall tabanlıysa mevcut Hall durumundan doğru adım seçilir; sensorless ise hizalama ve yumuşak açık çevrim rampası uygulanır. Akım, bus gerilimi, fault ve RPM loglanır. Titreşim, ters yön, aşırı akım veya düzensiz ses varsa hemen durdurulur. İlk hedef hız değildir: başarı kriteri motorun doğru yönde, sınırlı akımla, altı adımı kaybetmeden ve koruma mekanizmaları etkin hâlde dönmesidir.

Sonra geçiş sürelerinden RPM hesaplanır ve bağımsız bir takometre veya çalışan bir kartla karşılaştırılır. Start/stop sırasında duty bir anda sıçratılmaz, her döngüde sınırlı bir adımla artırılır: $D_{k+1} = \min(D_k + \Delta D, D_{\max})$. Aynı yaklaşım hedef RPM rampası için de kullanılır. Ani yön değişimi yüksek akım ve mekanik zorlanma oluşturur; önce hız güvenli seviyeye düşürülür. Kapalı çevrimde önce yalnız P ile küçük kazançtan başlanır, sistem kararlı

olduğunda kalıcı hatayı azaltmak için I eklenir. Her değişiklikte hedef RPM, ölçülen RPM, duty-cycle, faz veya DC-bus akımı, fault bayrakları, başlangıç ve yerleşme süresi kaydedilir.

Aşama 8 · Hata testleri

TEST	BEKLENEN DAVRANIŞ	KANIT
Enable kapatılması	Tüm güç çıkışları güvenli seviyeye geçer	Osiloskop görüntüsü
Driver fault	PWM donanımsal kapanır, fault kaydedilir	Fault pini ve PWM birlikte
Hall kablosu çıkarma	Geçersiz kod veya zaman aşımı ile durur	Log ve hata kodu
Rotor kilitlenmesi	Akım veya komütasyon zaman aşımı koruması çalışır	Akım grafiği, fault
Düşük bus gerilimi	Kontrollü stop veya startup engeli	Gerilim ve state logu
Aşırı hız	Duty kesilir veya kontrollü duruş	RPM ve state logu
Watchdog	Yazılım kilitlenmesinde güvenli reset ve çıkış kapanması	Reset sebebi

Hata ayıklama: belirti → olası neden → kontrol

Motor dönmediğinde tahminle parametre değiştirmek zaman kaybıdır.

BELİRTİ	OLASI NEDENLER	KONTROLLÜ İNCELEME
Motor yalnız titriyor	Faz sırası yanlış; Hall tablosu yanlış; başlangıç torku yetersiz; bir faz çalışmıyor	Altı state'i logic analyzer ile incele; faz-Hall eşlemesini yeniden çıkar; akım limitini kontrol et
Motor ters dönüyor	Komütasyon sırası ters; iki faz yer değişmiş	Yazılım sırasını kontrollü tersle; sensör eşlemesini doğrula
Başlangıçta yüksek akım	Shoot-through; yanlış state; çok yüksek duty; rotor kilitli	Dead-time ve gate dalga şekillerini ölç; güç vermeden tabloyu doğrula; duty'yi sınırla
Boşta dönüyor, yükte kalkmıyor	Startup rampası hızlı; akım limiti düşük; besleme zayıf; komütasyon gecikmesi yanlış	Bus gerilimi ve akımı logla; rampayı yumuşat; üretici sınırları içinde parametre ayarla

BELİRTİ	OLASI NEDENLER	KONTROLLÜ İNCELEME
Sensorless senkron kaybediyor	Zero crossing gürültüsü; demanyetizasyon maskesi kısa; 30° gecikme yanlış;	Boş faz dalga şeklini ölç; ADC tetikleme anını ve filtreyi doğru; hız aralığına göre gecikmeyi güncelle
RPM sıcıyor	Capture taşması; Hall bounce/gürültü; tek periyot ölçümü; yanlış kutup çifti	Zaman damgası logla; geçersiz kısa aralıkları reddet; hareketli ortalama kullan; p değerini doğru
MOSFET ısınıyor	Dead-time az veya fazla; yavaş gate sürme; yüksek anahtarlama kaybı; sürekli yüksek akım	Gate dalga şekli, sıcaklık ve akımı ölç; driver/MOSFET datasheet'ini incele
MCU reset oluyor	Besleme çökmesi; EMI; ground/layout problemi; watchdog	Reset sebebini kaydet; logic ve güç beslemelerini ölç; decoupling ve bağlantıları incele

AYNI ZAMAN EKSENİNDE DÖRT SİNYAL

1 · high-side PWM 2 · aynı kol low-side 3 · Hall / BEMF olayı 4 · akım / fault

Sinyalleri ayrı ayrı görmek "önce hangisi oldu?" sorusunu cevaplamaz. Dördü birlikte kaydedilirse komütasyonun geç mi kaldığı, PWM'in çakışıp çakışmadığı, fault'un önce mi geldiği zaman ekseninde cevaplanır.

OSİLOSKOP GROUND KLİPSİ UYARISI

Toprak referanslı standart probun ground klipsini inverter'ın U-V-W switching node'una bağlamak kısa devre ve ekipman hasarı oluşturur. Yüksek taraf veya faz-faz ölçümü için uygun diferansiyel prob, izolasyon yöntemi ve laboratuvar prosedürü kullanılır. Kondansatörlerin boşaldığı ölçülmeden karta dokunulmaz.

Log tasarımı: gerçek zamanlı kontrolü bozmamak için ring buffer kullanılır — hızlı ISR yalnız örneği belleğe yazar, UART aktarımı daha yavaş görevde yapılır.

Güvenlik ve koruma tasarımı

Koruma iki katmandır: donanım hızlı yolu olayı milisaniyeden önce keser, yazılım ise sebebi anlar ve kaydeder. İkisi birbirinin yerine geçmez — interrupt gecikmesi bile MOSFET için uzun olabilir.

KORUMA	DONANIMSAL HIZLI YOL	YAZILIM GÖREVİ
Aşırı akım	Comparator + BKIN/fault ile PWM kesme	Sebebi kaydetme, yeniden başlatmayı engelleme
Gate driver fault	Driver çıkışı PWM brake girişine bağlanır	Fault register'larını okuma ve raporlama
Yüksek/düşük bus gerilimi	Sürücüde UVLO/OVLO varsa hızlı koruma	ADC ile eşik ve zaman filtresi
Aşırı sıcaklık	Termal shutdown veya comparator	Sıcaklık trendi, derating ve kontrollü stop

KORUMA	DONANIMSAL HIZLI YOL	YAZILIM GÖREVİ
Stall	Akım limiti katkı sağlar	Hız yok + yüksek duty/akım koşulunu tanıma
Watchdog	Bağımsız watchdog reseti	Periyodik besleme ve reset sebebi kaydı

GÜVENLİ BAŞLANGIÇ VARSAYIMLARI

MCU reset olurken PWM pinleri floating kalıp sürücüyü açmamalı · enable pini harici pull ile güvenli varsayılan seviyede tutulmalı · yazılım başlamadan fault pininin geçerli olduğu doğrulanmalı · akım sensörü offset kalibrasyonu motor enerjisizken yapılmalı · geçersiz Hall kodu veya beklenmeyen timeout çıkışları kapatmalı · debugger breakpoint'inde timer çıkışlarının ne yaptığı bilinmeli.

MEKANİK GÜVENLİK

Motor sağlam bir düzeneğe sabitlenmeli · ilk testte pervane, kesici veya ağır dengesiz yük bulunmamalı · gevşek kablolar dönen parçadan uzak tutulmalı · koruyucu gözlük ve laboratuvar prosedürü kullanılmalı · motorun nominal hız ve akım sınırları aşılmamalı.

REJENERATİF ENERJİ · MOTOR JENERATÖRE DÖNÜŞÜNCÜ

Motor yavaşlarken jeneratör gibi davranır ve enerjiyi DC bus'a geri gönderir: yavaşlayan motor → inverter → bus gerilimi ↑. Her laboratuvar güç kaynağı bu enerjiyi ememez. Ani stop veya aktif fren uygulamadan önce sürücünün frenleme yöntemi, bus kondansatörü, koruma eşiği ve gerekiyorsa fren direnci incelenir.

Proje yapısı ve gösterilebilir çıktılar

ÖNERİLEN KLASÖR AYRIMI

motor_control	üst seviye API: start, stop, hedef RPM
motor_state_machine	durum geçişleri ve fault kararları
commutation	six-step tablosu · donanımdan bağımsız
speed_control	rampa ve PI
motor_fault	fault bayrakları ve kayıt
motor_hw_stm32 / _esp32	timer, ADC, GPIO · tek değişken katman
Tests · Docs	tablo testleri, pin_mapping, test_plan, dalga şekilleri

ÖN KANIT

- 01 Motor ve sürücü parametre tablosu
- 02 Sistem blok diyagramı ve pin eşleme tablosu
- 03 Altı durum komütasyon tablosu
- 04 Complementary PWM ve dead-time osiloskop görüntüsü
- 05 Hall veya Back-EMF geçiş görüntüsü
- 06 Başlangıç durum makinesi diyagramı
- 07 Hedef ve ölçülen RPM grafiği

08 Fault testi sonuç tablosu

09 Kod modülleri ve README

10 Bilinen sınırlamalar ve sonraki geliştirmeler

"Motor döndü" bir sonuç değil, bir gözlemdir. Bu on kanıt işin mühendislik tarafını gösterir.

ÖNCELİK SIRASI

Önce güvenli PWM, sonra doğru altı durum, ardından geri besleme, en son hız kontrolü. FOC, Clarke-Park ve SVPWM bu proje gerçekten six-step istiyorsa ilk hedef değildir.

EK A

Hızlı referans, formüller ve çalışma planı

Six-step hızlı tablo

ADIM	KAYNAK	DÖNÜŞ	BOŞ / ÖLÇÜLEN
1	U	V	W
2	U	W	V
3	V	W	U
4	V	U	W
5	W	U	V
6	W	V	U

Bu tablo faz akışını anlatır. Sürücü pinlerine uygulanacak gerçek high/low kombinasyonu, giriş polaritesi ve PWM modu datasheet truth table'ından çıkarılır.

Temel formüller

$$V = I \cdot R \cdot \text{Ohm kanunu}$$

$$P_{DC} = V \cdot I \cdot \text{aktarılan güç}$$

$$P_{direnç} = I^2 \cdot R \cdot \text{ısı kaybı}$$

$$D = t_{on} / T \cdot \text{duty-cycle}$$

$$\theta_e = p \cdot \theta_m \cdot \text{elektriksel/mekanik açı}$$

$$RPM = 60 / (6 \cdot p \cdot \Delta t) \cdot \text{sektör süresinden hız}$$

$$RPM = (\Delta count / N_{tur}) \times (60 / \Delta t) \cdot \text{encoder}$$

$$f_{PWM,center} = f_{TIM} / [2(PSC+1)(ARR+1)]$$

$$V_{ADC} = ADC_{raw} / (2^N - 1) \cdot V_{ref}$$

$$I = (V_{ADC} - V_{offset}) / (G \cdot R_{shunt})$$

Kart geldiğinde sorulacak 15 soru

01 Motorun tam modeli nedir?

02 Nominal gerilim ve sürekli akım kaçtır?

03 Kutup çifti sayısı kaçtır?

04 Motorda Hall veya başka konum sensörü var mı?

05 Yalnızca U-V-W mi çıkıyor?

06 Sürücü kartının tam modeli ve şeması nedir?

07 Kart hazır inverter mi, gate driver mı, ESC mi?

08 Sürücü 3-PWM mi, 6-PWM mi, başka komut mu bekliyor?

09 Logic girişleri 3.3 V uyumlu mu, aktif-high mı?

10 Dead-time sürücüde mi MCU'da mı oluşacak?

11 Fault pini hangi hatalarda aktif olur?

12 Akım ölçümü ve ölçeceği nedir?

13 Back-EMF ölçüm devresi kartta var mı?

14 İlk test için izin verilen bus gerilimi ve akım limiti nedir?

15 Beklenen teslimat yalnız dönmesi mi; RPM/yön/fault kontrolü de var mı?

İki haftalık örnek çalışma planı

GÜN	KONU	SOMUT ÇIKTI
1	BLDC, stator, rotor, U-V-W ve six-step	Kavram özeti ve altı durumu elle çizme
2	Yarım köprü, inverter, gate driver, dead-time	Güç ve kontrol yollarını açıklayabilme
3	Motor ve sürücü datasheet'i	Donanım kimlik kartı ve pin eşleme tablosu
4	STM32 TIM1/TIM8 veya ESP MCPWM	Güç kapalıyken üç senkron PWM
5	Complementary PWM, dead-time, fault	Osiloskop kanıtı ve güvenli kapatma
6	Komütasyon lookup table	Altı adımı düşük hızda logic analyzer ile gösterme
7	Hall veya Back-EMF geri beslemesi	Geçiş sırası ve zaman damgası kaydı
8	RPM hesabı ve filtre	Elle döndürme veya düşük hız testinde RPM
9	Düşük enerjili ilk dönüş	Sınırlı akımda kararlı dönüş
10	Sensorless ise startup rampası	ALIGN → OPEN LOOP → CLOSED LOOP geçişi
11	Hız rampası ve temel PI	Hedef-ölçülen RPM grafiği
12	Yön ve kontrollü duruş	Güvenli forward/reverse senaryosu
13	Fault senaryoları	Test sonuç tablosu
14	Temizlik ve dokümantasyon	Demo, README ve teknik sunum

EK B

Terimler sözlüğü

BLDC · Fırçasız doğru akım motoru; elektronik komütasyon gerekir.

Stator · Motorun sabit, sargıları taşıyan kısmı.

Rotor · Dönen, çoğunlukla kalıcı mıknatıslı kısım.

Faz · U, V, W olarak adlandırılan sargı bağlantılarından biri.

Komütasyon · Hangi faz çiftinin enerjilendiğini doğru sırada değiştirme işlemi.

Six-step · Bir elektriksel turu altı 60° sektöre bölen trapezoidal kontrol.

Elektriksel açı · Manyetik alanın bir elektriksel çevrimdeki açısı; p ile mekanikten ayrılır.

PWM · Dijital anahtarlama ile ortalama gerilim/güç kontrol yöntemi.

Duty-cycle · PWM periyodunda sinyalin açık kaldığı oran.

High-side · Yarım köprünün DC+ tarafındaki güç anahtarı.

Low-side · Yarım köprünün GND tarafındaki güç anahtarı.

Shunt · Akım ölçmek için kullanılan düşük değerli hassas direnç.

ADC · Analog gerilimi sayısal değere dönüştüren çevre birimi.

DC-link · İnverterin DC besleme barası ve üzerindeki enerji depolayan kondansatörler.

Dead-time · Aynı koldaki iki anahtar arasında bırakılan çakışmasız bekleme.

Shoot-through · High ve low side'in birlikte açılmasıyla oluşan DC-bus kısa devresi.

Gate driver · MOSFET kapılarını uygun gerilim ve akımla süren entegre.

İnverter · DC enerjiyi anahtarlایarak üç fazlı değişken gerilim üreten güç katı.

Hall sensörü · Manyetik alanı algılayıp rotor sektörünü dijital olarak bildiren sensör.

Back-EMF · Dönen motorun sargılarında indüklenen, hız/konum bilgisi taşıyan gerilim.

Zero crossing · Boştaki fazın Back-EMF sinyalinin referans eşiğini geçtiği olay.

Sensorless · Hall veya encoder olmadan faz gerilim/akımından konum tahmini.

Open loop · Geri beslemeye dayanmadan önceden belirlenen zamanlamayla kontrol.

Closed loop · Ölçülen hız/konumu hedefle karşılaştırıp komutu düzelten kontrol.

PI controller · Oransal ve integral terimlerle hatadan çıkış üreten denetleyici.

Anti-windup · Çıkış sınırdarken integral teriminin birikmesini engelleyen önlem.

Input capture · Bir dijital kenarın oluştuğu timer sayım değerini donanımda kaydetme.

BKIN / brake · Fault'ta PWM çıkışlarını donanımsal olarak güvenli duruma getiren giriş.

Quadrature · Encoder A ve B kanalları arasındaki 90° faz farkı düzeni; yön bilgisinin kaynağı.

HAL · Hardware Abstraction Layer; ST'nin çevre birimlerini soyutlayan C kütüphanesi.

Push-pull · Pinin hem 3.3 V'a hem GND'ye aktif sürebildiği çıkış modu.

Yüksek empedans · Pinin sürülmediği, boştaki durum (FLOAT / input modu).

LPUART · Low-Power UART; asenkron seri haberleşme birimi.

8-N-1 · 8 veri biti, parity yok, 1 stop biti.

SWD · Serial Wire Debug; iki telli (SWDIO, SWCLK) programlama/debug arayüzü.

VCP · Virtual COM Port; ST-LINK'in UART'ı USB üzerinden seri port gibi göstermesi.

USB CDC · MCU'nun kendi USB birimiyle sanal seri port sunduğu cihaz sınıfı.

DFU · Device Firmware Upgrade; USB üzerinden firmware yazma bootloader modu.

HSI / PLL · Dahili osilatör ve onu sistem saatine çarpan faz kilitli döngü.

SysTick · Cortex-M çekirdek zamanlayıcısı; HAL_GetTick() ms sayacını besler.

.ioc / .elf · CubeMX proje dosyası; derlenmiş firmware dosyası.

Ek C · Kaynaklar

Bu kitaptaki teknik yaklaşım üretici kaynaklarıyla uyumludur. Kesin pin, register, elektriksel sınır ve konfigürasyon için kullanılan parça numarasına ait güncel datasheet, reference manual ve errata esas alınır.

- 1 · STMicroelectronics — STM32 Motor Control, 6-step Firmware Algorithm (wiki.st.com): sensorless Back-EMF algılama, zero crossing, $\sim 30^\circ$ gecikme, Hall ve six-step bileşenleri.
- 2 · STMicroelectronics — AN4220 uygulama notu ve ilgili six-step referans dokümanları.
- 3 · Kullanılan gate driver / motor driver entegresinin datasheet'i: giriş doğruluk tablosu, dead-time, fault davranışı.
- 4 · Seçilen MCU'nun reference manual'ı: timer, complementary PWM, break girişi, ADC tetikleme ve capture bölümleri.
- 5 · Motor üreticisinin veri sayfası: kutup çifti sayısı, Hall yerleşimi, faz sırası, akım ve hız sınırları.
- 6 · Bu kitabın kaynak dokümanları: görsel anlatım rehberi, teknik eğitim notları 01–02 ve NUCLEO-G491RE çalışma dokümanı.

Bu kitap eğitim amaçlıdır ve kullanılan motorun, sürücünün veya mikrodenetleyicinin datasheet'inin yerini tutmaz. Her pin, register ve elektriksel sınır, elinizdeki tam parça numarasına ait güncel belgeyle doğrulanmalıdır.